



Introduction to Git & GitLab

[\(GRAEME.GRIMES@ED.AC.UK\)](mailto:GRAEME.GRIMES@ED.AC.UK)

[HTTPS://GIT.ECDF.ED.AC.UK/GGRIMES2/GIT4IGC](https://git.ecdf.ed.ac.uk/GGRIMES2/GIT4IGC)

Rough Timetable

- ▶ Break 11
- ▶ Lunch 12:30
- ▶ Start 1:30
- ▶ Finish 4:00

Tracking changes



From: <http://www.phdcomics.com/comics/archive.php?comiciid=1531>

What is git?

- ▶ Also known as a *revision control* or *source control system*
 - ▶ There are others: SVN, mercurial, ...
- ▶ Records & preserves history of changes to files
 - ▶ Think of a **lab book for code**
- ▶ Not just for source code:
 - ▶ Configuration files
 - ▶ Parameter sets
 - ▶ Data files (**git lfs** for large files)
 - ▶ User documentation & manuals
 - ▶ Conference/journal papers/book chapters

Why use version control system?

- ▶ Keeping files synchronised across different systems
 - ▶ e.g. home PC, your laptop, work PC, Eddie, ...

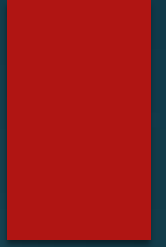
Why use version control system?

- ▶ Keeping files synchronised across different systems
- ▶ Want to know the provenance of your files
 - ▶ Need to check previous code version
 - ▶ Who did what/when?

Why use version control system?

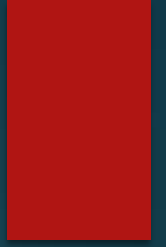
- ▶ Keeping files synchronised across different systems
- ▶ Want to know the provenance of your files
- ▶ Back-up precious files
 - ▶ recover previous states of the system
 - ▶ Deleted files
 - ▶ State of the system when you published a paper/report
 - ▶ change jobs – take your code with you

Why use version control system?



- ▶ Keeping files synchronised across different systems
 - ▶ e.g. home PC, your laptop, work PC, Eddie, ...
- ▶ Want to know the provenance of your files
 - ▶ Need to check previous code version
 - ▶ Who did what/when?
- ▶ Back-up precious files
- ▶ You want to collaborate – gives you a platform that facilitates this

Why use version control system?



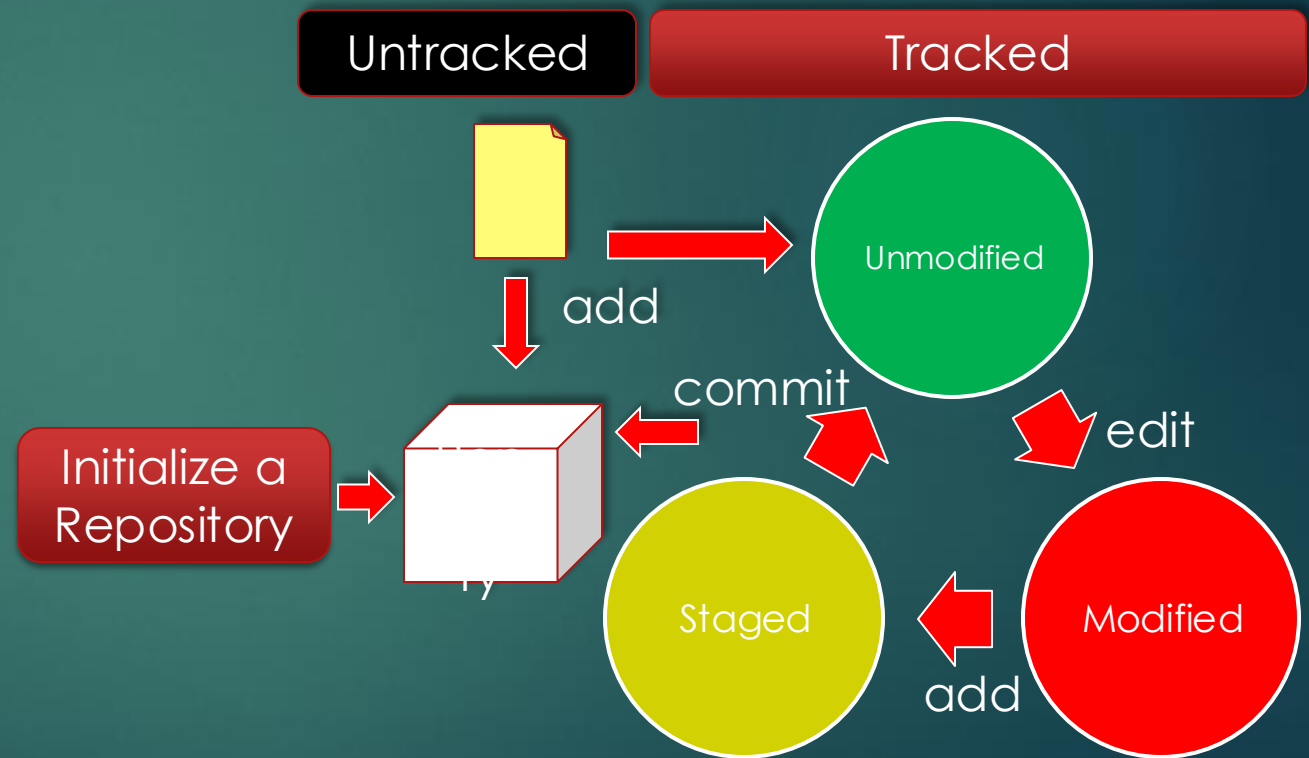
- ▶ Keeping files synchronised across different systems
 - ▶ e.g. home PC, your laptop, work PC, Eddie, ...
- ▶ Want to know the provenance of your files
 - ▶ Need to check previous code version
 - ▶ Who did what/when?
- ▶ Back-up precious files
- ▶ You want to collaborate – gives you a platform that facilitates this
- ▶ Show off your work

What we will cover

1. Tracking changes with a local repository
2. Working with a remote repository GitLab
3. Collaborating with colleagues using a repository
4. A little on branching

what are we going to do?

- ▶ Tell git who you are (only do this once per machine)
 - ▶ your name
 - ▶ your email address
- ▶ Initialise a repository (only do this once)
- ▶ You add (register files to your repo)
- ▶ Files will be in one of the following states
 - ▶ **untracked** - git does not know about them
 - ▶ **tracked** – you have added them
 - ▶ **unmodified** – files are unchanged
 - ▶ **modified** – you modified a file
 - ▶ **staged** – about to add to the repository



Start

Git on eddie

```
$ ssh <user>@eddie.ecdf.ed.ac.uk
```

```
$ qlogin
```

```
$ git --version
```

Configuring git

.GITCONFIG

git config

▶ Purpose:

- ▶ Configure user-specific settings, like email, name, and preferred editor.
- ▶ Define behavior of a repository, a user, or the entire system.

▶ Levels:

- ▶ **Local**: Specific to a single repository (**--local**).
- ▶ **Global**: User-wide, applies to all repos for that user (**--global**).

Git config ~/.gitconfig

- ▶ Git stores most of the preferences in a global configuration file called `.gitconfig`. It is located in `~/.gitconfig`.

```
$ git config --global user.name "Graeme Grimes"
```

```
$ git config --global user.email Graeme.Grimes@ed.ac.uk
```

```
$ git config --global init.defaultBranch main
```

Other useful config settings

```
$ git config --global color.ui "auto" # Git will automatically color its output when  
outputting to a terminal.
```

```
$ git config --global core.editor "nano -w" # Disable word wrap
```

```
$ git config --global help.autocorrect 1 # Git When a user mistypes a Git command,  
Git will suggest the closest match.
```

```
$ git config --global pull.rebase false # Sets the default way of handling merges
```

Creating a repository

`GIT INIT`

git init

```
$ mkdir MyProject
```

```
$ cd MyProject
```

```
$ git status
```

```
$ git init
```

```
$ git status
```

```
$ nano Readme.md
```

```
# My eddie Python project
```

git status

Repository

Staging Area

Working directory

README.md

```
# My eddie Python project
```

```
$ git add Readme.md
```

Repository

Staging Area

README.m
d

Working directory

README.m
d

```
# My eddie Python project
```

```
git commit -m "Add README.md"
```

Repository

README.md
23bd868

Staging Area

Working directory

README.m
d

```
# My eddie Python project
```

Git messages

1. Limit the Subject Line: Keep the subject or summary line to 50 characters or fewer. This ensures it's easily readable in various Git tools.
2. Use the Imperative Mood: Frame your commit message as a command or action, e.g., "Fix" rather than "Fixed" or "Fixes."
3. Avoid Vague Messages: Instead of writing "Update code" or "Fix bug", specify what you did, such as "Add error handling for login process."

Keeping these rules in mind ensures your commit history remains a useful and navigable resource for understanding the evolution of your project.

git log

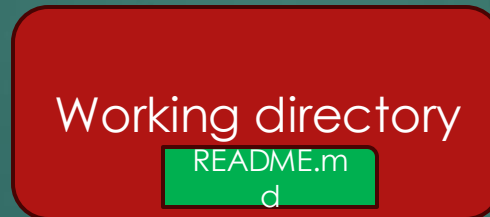
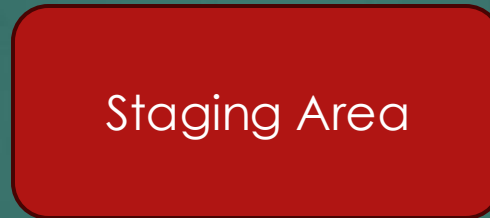
- Show the sequence of commits starting from the current one, in reverse chronological order of the Git repository in the current working directory:

```
git log  
  
commit f00b5b49b39884a9bf978bc272c82d862976abcb (HEAD -> main)  
Author: Graeme Grimes <Graeme.Grimes@ed.ac.uk>  
Date:   Tue Sep 24 10:06:58 2024 +0100
```

```
Add README.md
```

HEAD

The HEAD is a pointer to where you are in the commit history



```
# My eddie Python project
```

Exercise

- ▶ Add your commit id to the etherpad

make changes to README.md



```
# My eddie Python project
To run Python on eddie need to run:
    module load igmm/apps/python/3.12.3
```

git add README.md



```
# My eddie Python project
To run Python on eddie need to run:
module load igmm/apps/python/3.12.3
```

make changes to README.md



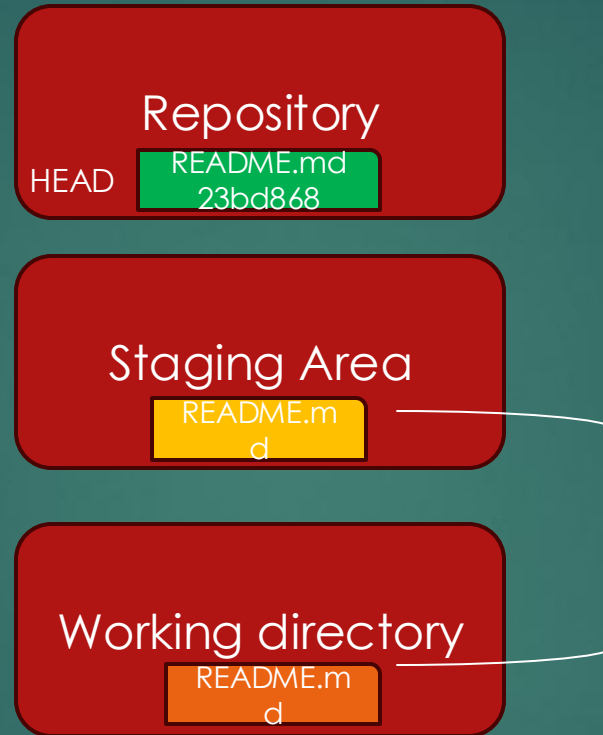
```
# My eddie Python project
To run Python on eddie need to run:
module load igmm/apps/python/3.12.3
## Run Script
```

git diff README.md

Changes you made staging area
for the next commit.

In other words, the differences are
what you **could** tell Git to
further add to the staging area but
you still haven't.

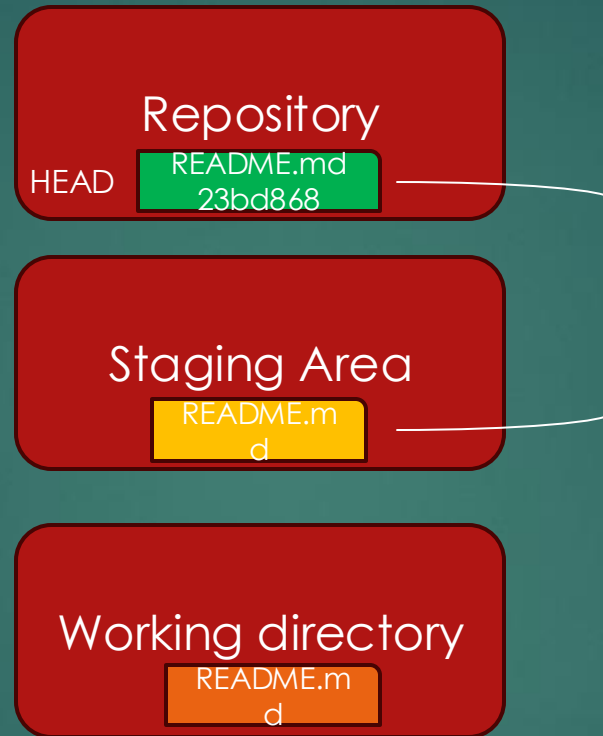
You can stage these change



```
# My eddie Python project
To run Python on eddie need to run:
module load igmm/apps/python/3.12.3
## Run Script
```

3.3

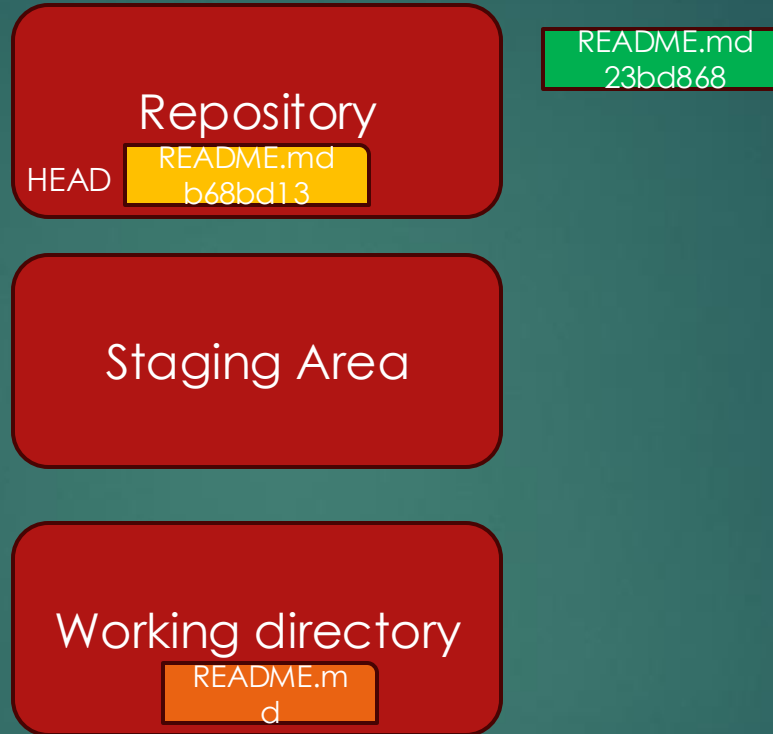
git diff --staged README.md



```
# My eddie Python project
To run Python on eddie need to run:
module load igmm/apps/python/3.12.3
## Run Script
```

3.12.3

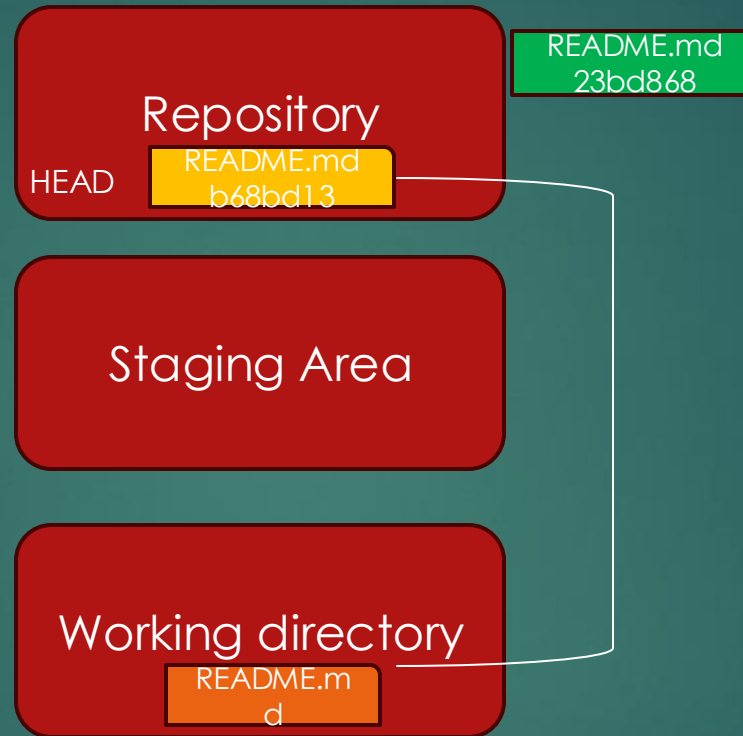
git commit -m "Update README.md"



```
# My eddie Python project
To run Python on eddie need to
run:
    module load
igmm/apps/python/3.7.3
## Run Script
```

3.7.3

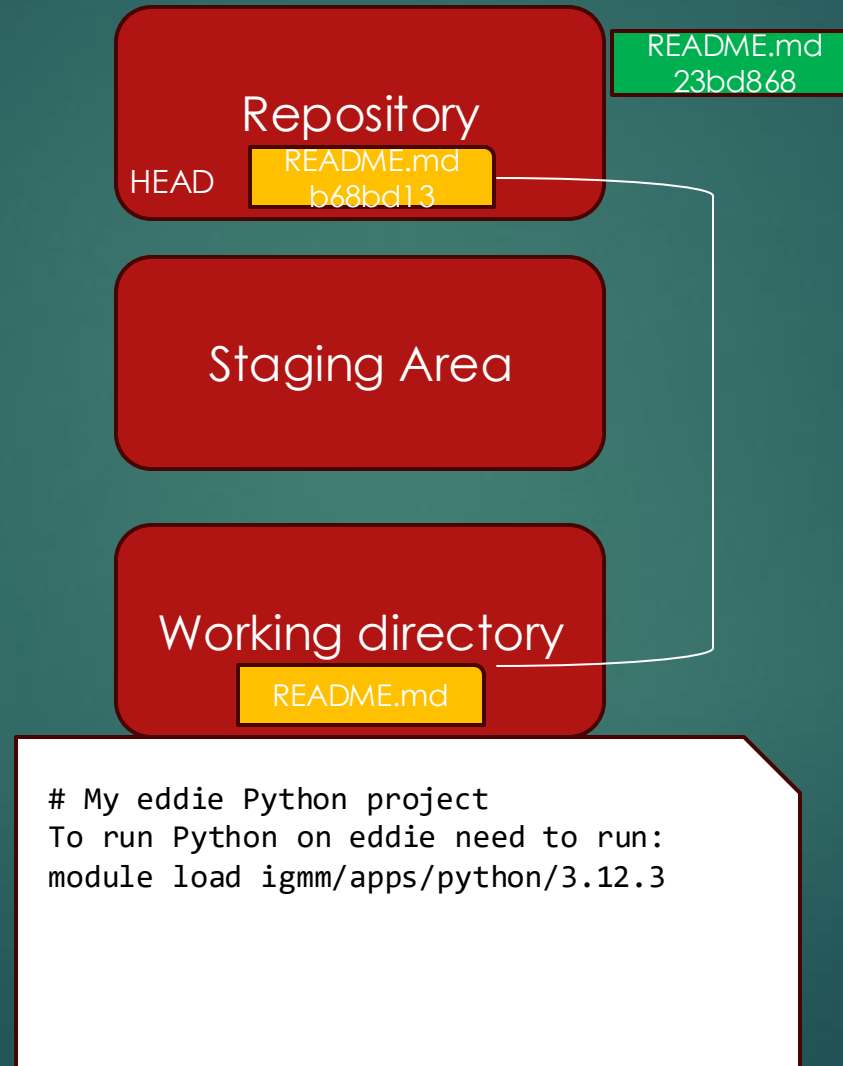
git diff HEAD README.md



```
# My eddie Python project
To run Python on eddie need to run:
module load igmm/apps/python/3.12.3
## Run Script
```

3.12.3
3.7.3

Restore file: `git restore Readme.md`



Be careful this will delete the change in working directory

Directories

Adding Directories

- ▶ Cannot add directories by themselves:

```
$ mkdir Src
```

```
$ git status
```

```
$ git add Src
```

```
$ git status
```

Exercise: Add code and commit

- ▶ you need to add a file (sometimes `.gitkeep`) - so let's add a file:

```
$ nano Src/MyScript.py
```

Src/MyScript.py

```
#!/usr/bin/env python
```

```
print("Hello World!")
```

Adding data

```
$ mkdir Data
```

```
$ cp /exports/igmm/software-  
rl9/bac/dataWranglingGenomics/20230529/workshop_data/combined_tidy_vcf.csv  
Data/data.csv
```

```
$ head Data/data.csv
```

```
$ git status
```

```
$ git add Data/data.csv      # Comment on when to commit data
```

```
$ git commit -m "Suitable commit message"
```

Let us modify our Python script

Src/MyScript.py

```
#!/usr/bin/env python

# load pandas library
import pandas as pd

# Read the data
variants = pd.read_csv("Data/data.csv")

# Check we have the data
print(variants)
```

Run Script

```
$ module load igmm/apps/python/3.7.3 # load python
$ python Src/MyScript.py # Make sure the code runs
$ git add Src/MyScript.py
$ git commit -m "Update MyScript.py by reading in data"
```

check the commits that you have made:

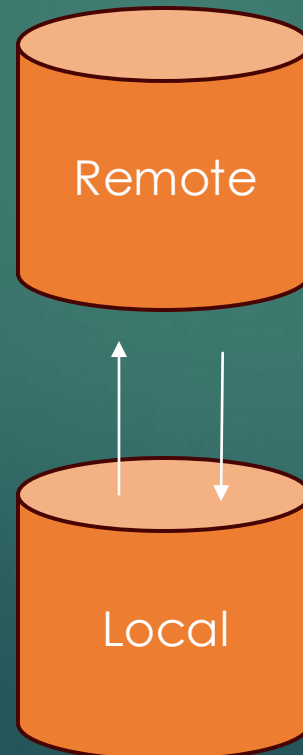
- ▶ `$ git log`
- ▶ `$ git log --oneline`

GIT Remotes

[HTTPS://GIT.ECDF.ED.AC.UK/](https://git.ecdf.ed.ac.uk/)

Git remote

- ▶ A git remote is a saved link to another copy of your repository, usually located on the internet.
- ▶ It lets you synchronize your local repository with that external one



remote git service providers

- ▶ University GitLab (<https://git.ecdf.ed.ac.uk/>)
- ▶ GitHub (<https://github.com>)
- ▶ Bitbucket (<http://bitbucket.org>)
- ▶ GitLab (<https://about.gitlab.com>)
- ▶ Launchpad (<https://launchpad.net>)
- ▶ SourceForge (<http://sourceforge.net>)
- ▶
- ▶ For more options see: <https://git.wiki.kernel.org/index.php/GitHosting>

https://git.ecdf.ed.ac.uk/



22

17

Search or go to...

Your work

Projects

Groups

Issues22

Merge requests>

To-Do List17

Milestones

Snippets

Activity

Your work / Projects

Projects

Explore projects

New project

Yours243

Starred2

Filter by name

Language

Updated date

All

Personal

G

igmmbioinformatics / **genomics_r**  Owner

☆ 0

👤 0

🔗 0

📄 0

Updated 1 hour ago

T

igmmbioinformatics / **trio-whole-exome**  Owner

✓

☆ 0

👤 0

🔗 1

📄 16

Updated 1 day ago

G

igmmbioinformatics / **Graeme Grimes**  Owner

☆ 0

👤 0

🔗 0

📄 10

Updated 1 day ago

I

igmmbioinformatics / **intro-data-science-teaching**  Owner

☆ 0

👤 0

🔗 0

📄 0

Updated 1 day ago

N

igmmbioinformatics / **nextflow_configs**  Owner

☆ 0

👤 0

🔗 0

📄 0

Updated 1 day ago

A

igmmbioinformatics / **Alison Meynert**  Owner

☆ 0

👤 0

🔗 0

📄 12

Updated 1 day ago



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Import project

Migrate your data from an external source like GitHub, Bitbucket, or another instance of GitLab.

You can also create a project from the command line. [Show command](#)



22

17

Search or go to...

Project

M

MyProject

Pinned

Issues

Merge requests

Members

Manage

Plan

Code

Secure

Deploy

Operate

Monitor

Analyze

Settings

Project 'MyProject' was successfully created.

M

MyProject

Star

0

Invite your team

Add members to this project and start collaborating with your team.

Invite members

The repository for this project is empty

You can get started by cloning the repository or start adding files to it with one of the following options.

Code

Upload File

New file

Add README

Add LICENSE

Add CHANGELOG

Add CONTRIBUTING

Add Wiki

Configure Integrations

Command line instructions

You can also upload existing files from your computer using the instructions below.

Git global setup

```
git config --global user.name "Graeme Grimes"
git config --global user.email "graeme.grimes@igmm.ed.ac.uk"
```

Create a new repository

```
git clone git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git
cd myproject
git switch --create main
touch README.md
git add README.md
git commit -m "add README"
git push --set-upstream origin main
```

Push an existing folder

```
cd existing_folder
git init --initial-branch=main
git remote add origin git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git
git add .
git commit -m "Initial commit"
git push --set-upstream origin main
```

Push an existing Git repository

```
cd existing_repo
git remote rename origin old-origin
git remote add origin git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git
git push --set-upstream origin --all
git push --set-upstream origin --tags
```

GitLab security

To enable password less access to your remote, you can add your public SSH Key from Eddie to the remote repository.

Generating a new SSH key

You can generate a new SSH key on your local machine. After you generate the key, you can add the public key to your account on GitLab to enable authentication for Git operations over SSH.

The University guide can be found [here](#)

Example below

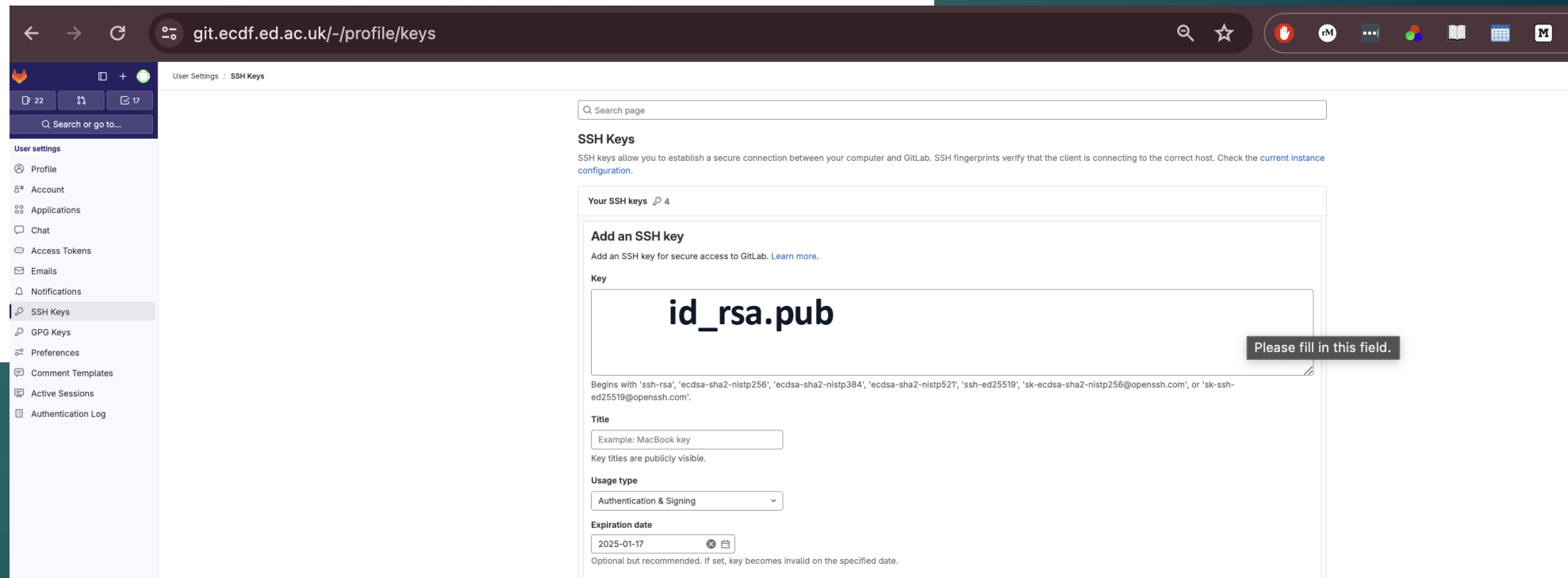
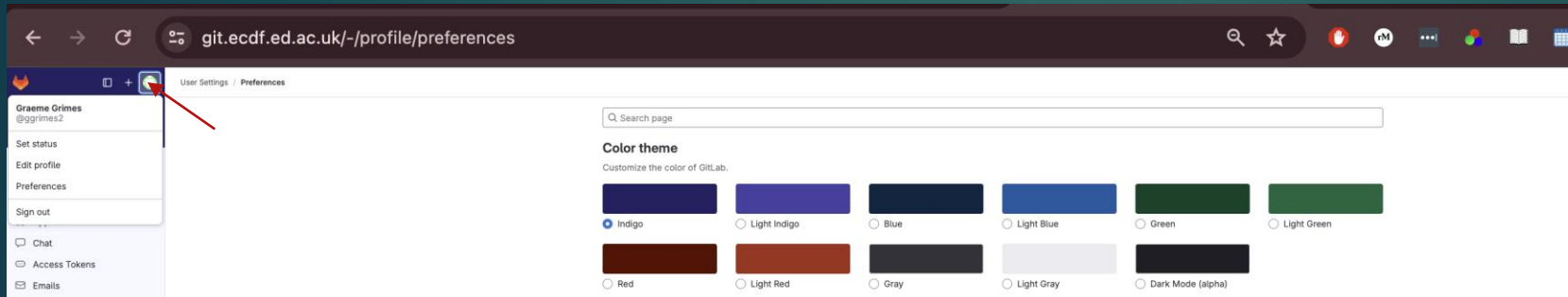
BASH < >

```
[username@login02(eddie) ~]$ ssh-keygen -o -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/username/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/username/.ssh/id_ed25519.
Your public key has been saved in /home/username/.ssh/id_ed25519.pub.
The key fingerprint is:
SHA256:3t/tZzvbviR5Xs1YYYQeaGtI2n02W(and some more characters)hxxt (email a
The key's randomart image is:
+---[ED25519 256]---+
| 00.. . 0... |
| 0. 0 0 ...0.+ |
| .+ + 0+ +0+ .0 |
| E 0 0. 0 +.B. . |
|          S . = 0 . |
|          . . . .+. |
|          . . 0.0+ |
|          . . =0= |
|          . .0@ |
+-----[SHA256]-----+
```

This will generate two files: the private key file (in this example `/home/username/.ssh/id_ed25519`) and the public key file (`/home/username/.ssh/id_ed25519.pub`) - it is vitally important that you never share your private key

You can now add the contents of the public key file `id_ed25519.pub` to the GitLab.

GitLab Security SSH Keys



Git remote (on eddie)

► \$ git remote --help

```
igctrain@node1c19:~/MyProject
GIT-REMOTE(1)                               Git Manual                               GIT-REMOTE(1)

NAME
    git-remote - manage set of tracked repositories

SYNOPSIS
    git remote [-v | --verbose]
    git remote add [-t <branch>] [-m <master>] [-f] [--[no-]tags] [--mirror=<fetch|push>] <name> <url>
    git remote rename <old> <new>
    git remote remove <name>
    git remote set-head <name> (-a | -d | <branch>)
    git remote set-branches [--add] <name> <branch>...
    git remote set-url [--push] <name> <newurl> [<oldurl>]
    git remote set-url --add [--push] <name> <newurl>
    git remote set-url --delete [--push] <name> <url>
    git remote [-v | --verbose] show [-n] <name>...
    git remote prune [-n | --dry-run] <name>...
    git remote [-v | --verbose] update [-p | --prune] [(<group> | <remote>)...]

Manual page git-remote(1) line 1 (press h for help or q to quit)
```

Git add origin

- ▶ `$ git remote add origin git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git`
- ▶ `$ git remote`
- ▶ `$ git remote -v`

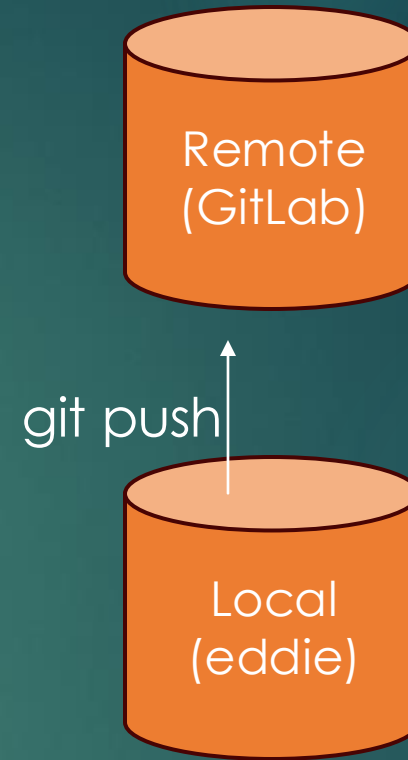
```
[igctrain@node1c19(eddie) MyProject]$ git remote --help
[igctrain@node1c19(eddie) MyProject]$ git remote add origin git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git
[igctrain@node1c19(eddie) MyProject]$ git remote
origin
[igctrain@node1c19(eddie) MyProject]$ git remote -v
origin  git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git (fetch)
origin  git@git.ecdf.ed.ac.uk:ggrimes2/myproject.git (push)
[igctrain@node1c19(eddie) MyProject]$
```

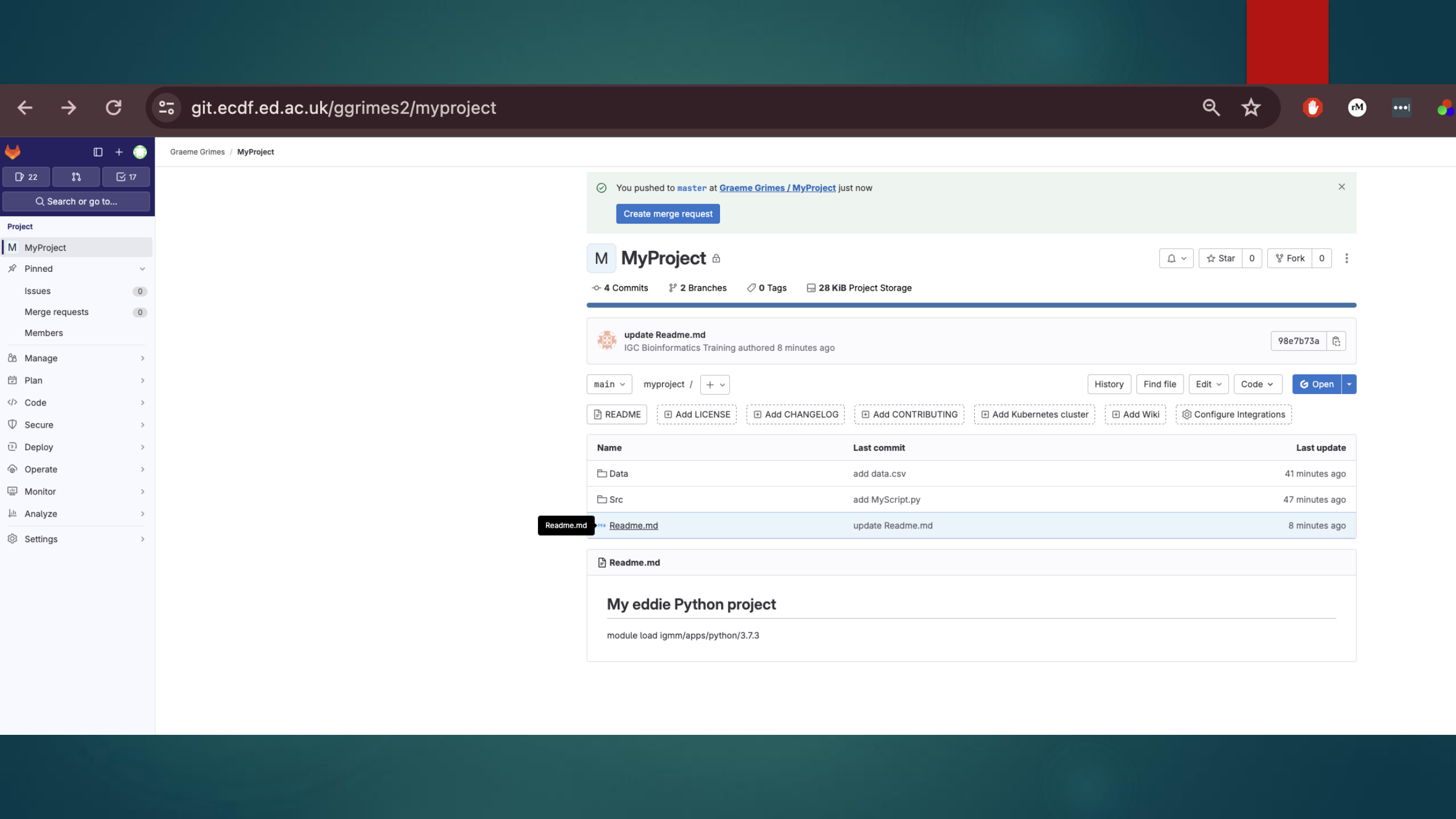
Origin

- ▶ The name "origin" for a git remote is not inherently special to Git's internal mechanics, but it is a widely accepted convention and has a few notable points:
- ▶ Default Name: When you clone a repository from a remote source using `git clone <repository_url>`, Git automatically names the remote repository "origin" by default. It's just a convention that Git follows.
- ▶ Widely Recognized: Because of this default behavior, many developers expect "origin" to refer to the primary or original repository from which the project was cloned. This shared understanding can help with clarity when collaborating.
- ▶ Can be Renamed or Removed: There's nothing binding about the name "origin." You can rename it or even have multiple remotes with different names to track different repositories or branches.
- ▶ In essence, "origin" is a convention, and while it's not mandatory to use, adhering to such conventions can make collaborative work smoother and more intuitive.

Git push

- ▶ Send local changes in the current branch to its default remote counterpart:
- ▶ `$ git push`





🐉 + 🌐

📄 22 🔁 17

🔍 Search or go to...

- Project
- M MyProject
 - 📌 Pinned ▾
 - 🔍 Issues 0
 - 🔄 Merge requests 0
 - Members
 - 👤 Manage >
 - 📅 Plan >
 - 🔗 Code >
 - 🛡️ Secure >
 - 🚀 Deploy >
 - ⚙️ Operate >
 - 📺 Monitor >
 - 📊 Analyze >
 - ⚙️ Settings >

Graeme Grimes / MyProject

✅ You pushed to **master** at [Graeme Grimes / MyProject](#) just now

Create merge request

M **MyProject** 🔒

🔍 4 Commits 🌿 2 Branches 🏷️ 0 Tags 📄 28 KiB Project Storage

🔔 ▾ ☆ Star 0 🍴 Fork 0 ⋮

🔧 **update Readme.md** 98e7b73a 🔄

IGC Bioinformatics Training authored 8 minutes ago

main ▾ myproject / + ▾

History Find file Edit ▾ Code ▾ Open ▾

📄 README ➕ Add LICENSE ➕ Add CHANGELOG ➕ Add CONTRIBUTING ➕ Add Kubernetes cluster ➕ Add Wiki ⚙️ Configure Integrations

Name	Last commit	Last update
📁 Data	add data.csv	41 minutes ago
📁 Src	add MyScript.py	47 minutes ago
📄 Readme.md	update Readme.md	8 minutes ago

📄 Readme.md

My eddie Python project

module load igmm/apps/python/3.7.3

Make Change to Readme.md (eddie)

```
$ nano Readme.md
```

Add

```
## Data
```

```
The  
[data](https://figshare.com/articles/dataset/Data_Carpentry_Ge  
nomics_beta_2_0/7726454?file=14632895) is made available under  
an [Creative Commons  
license](https://creativecommons.org/licenses/by/3.0/).
```

On Eddie

- ▶ `$ git status`
- ▶ `$ git add Readme.md`
- ▶ `$ git commit -m "Add Eddie and Data subsections"`
- ▶ `$ git status`
- ▶ `$ git push`

22

17

Search or go to...

Project

M

MyProject

Pinned

Issues

Merge requests

Members

Manage

Plan

Code

Secure

Deploy

Operate

Monitor

Analyze

Settings

Graeme Grimes / MyProject

MyProject

4 Commits2 Branches0 Tags28 KIB Project Storage

update Readme.md

Graeme Grimes authored just now

572363ad

main

myproject

History

Find file

Edit

Code

Open

README

Add LICENSE

Add CHANGELOG

Add CONTRIBUTING

Add Kubernetes cluster

Add Wiki

Configure Integrations

Name	Last commit	Last update
Data	add data.csv	42 minutes ago
Src	update Readme.md	just now
Readme.md	update Readme.md	just now

Readme.md

My eddie Python project

module load igmm/apps/python/3.7.3

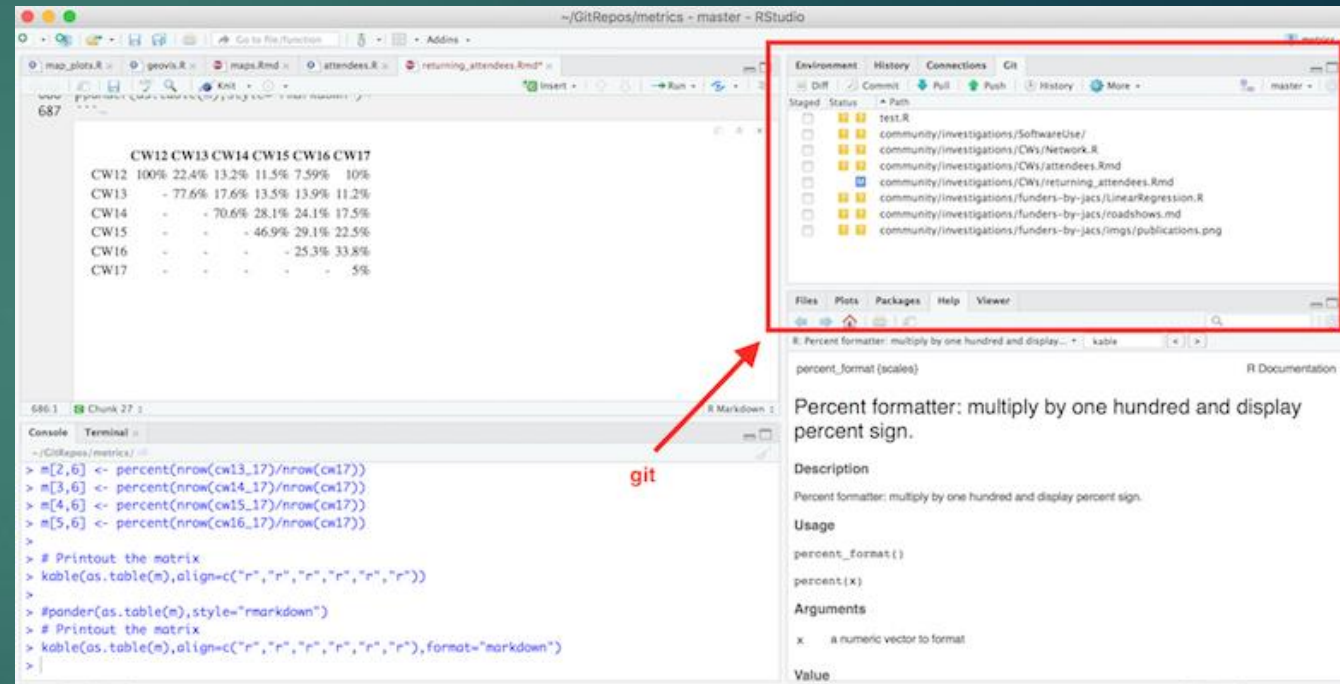
Data

The [data](#) is made available under an [Creative Commons license](#).

Git with a Graphical Interface

Can Use a GUI

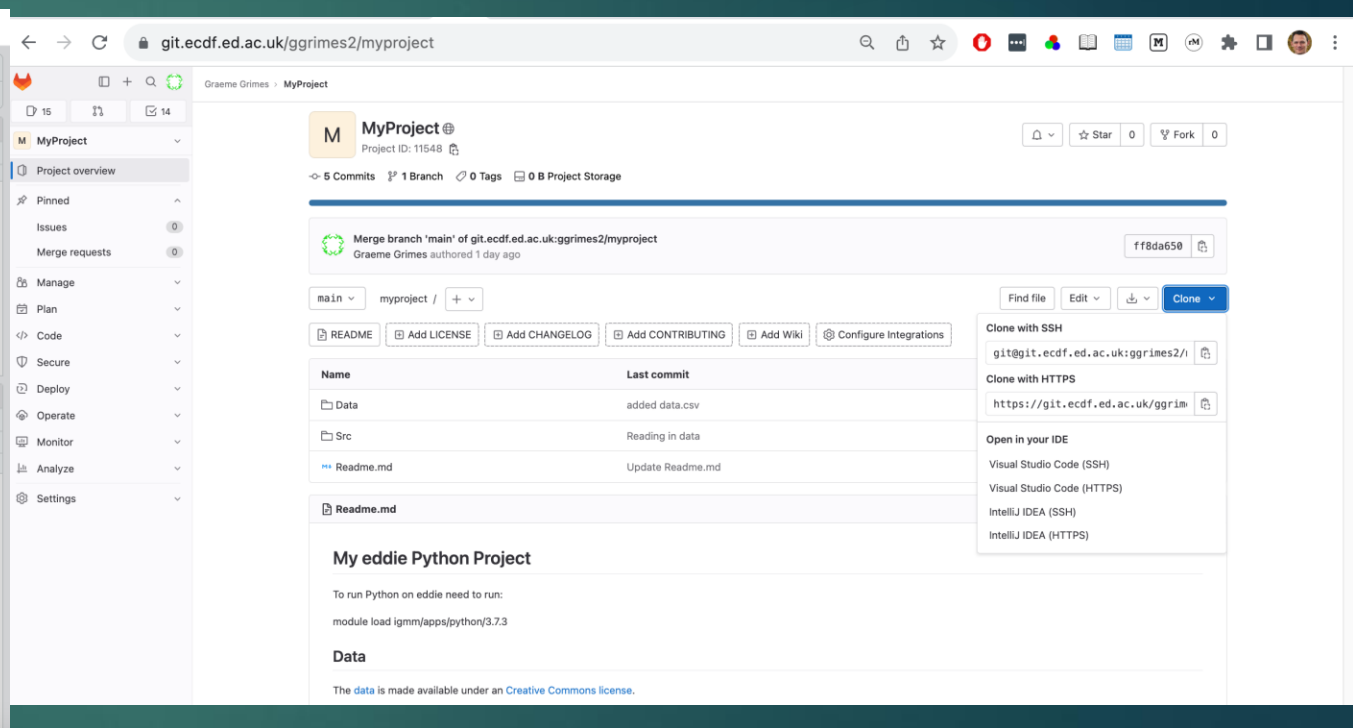
- List of free git GUIs
 - <https://git-scm.com/downloads/guis>
- Most IDEs provide support for git interactions
- BUT good to be able to use the shell
- All IDEs/GUIs use shell commands
 - Pain to use a GUI over a network
- You can still use a GUI locally
 - Use git on shell on remote machine
- Also easier: command line -> GUI

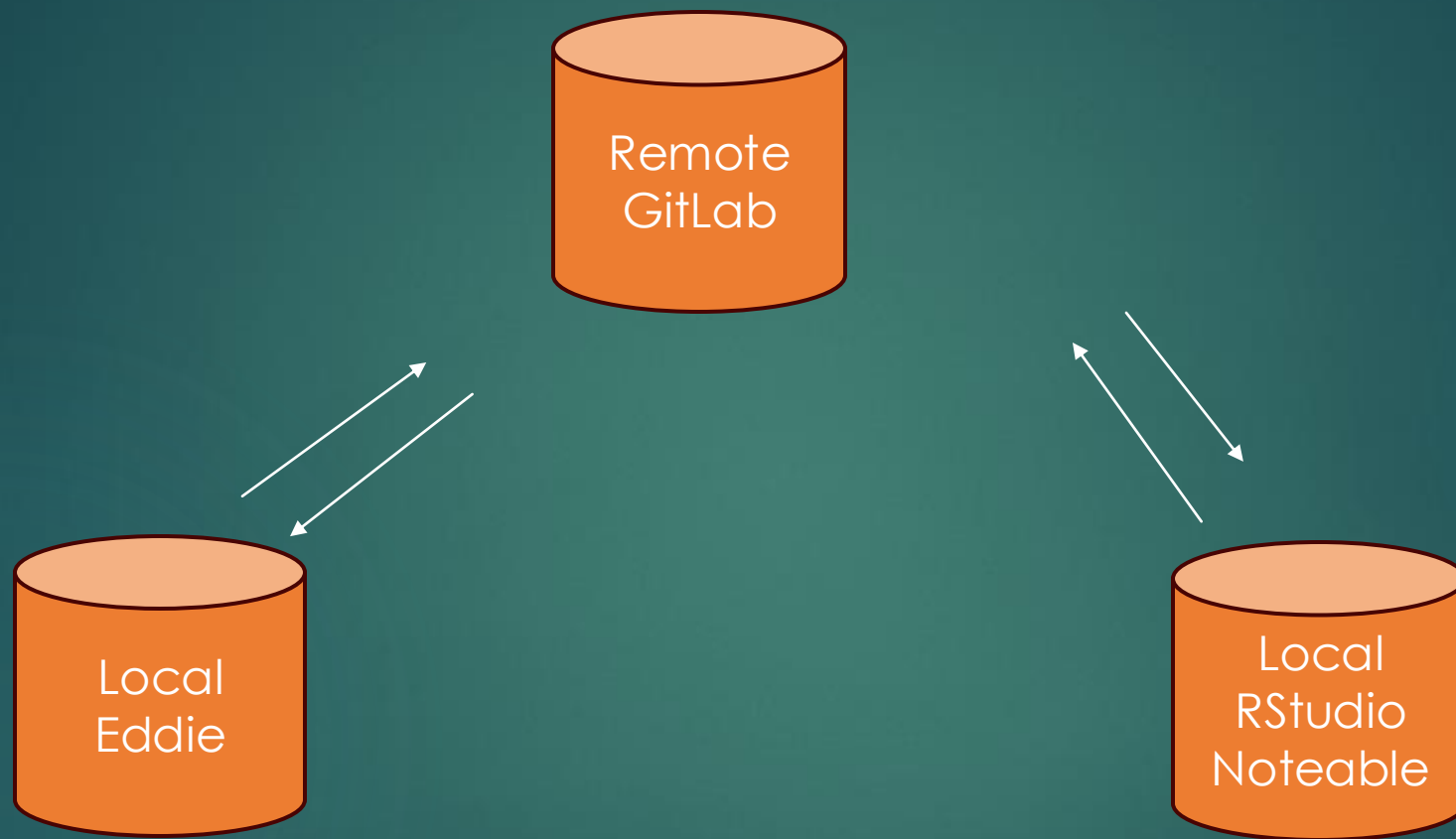




RStudio GUI

[HTTPS://NOTEABLE.EDINA.AC.UK/LOGIN](https://noteable.edina.ac.uk/login)





Git configuration

On each new computer need to configure git

```
$ git config -- global -- list
```

```
$ git --version
```

```
$ git config --global user.name "Graeme Grimes Noteable"
```

```
$ git config --global user.email "Graeme.Grimes@ed.ac.uk"
```

```
#also add pandas python library via pip
```

```
pip install pandas
```

Update Src/MyScript.py

The screenshot displays the RStudio web interface in a browser. The main editor window shows a Python script named `MyScript.py` with the following content:

```
1 #!/bin/env python
2
3 # load pandas
4 import pandas as pd
5
6 # Read data
7 variants = pd.read_csv("Data/data.csv")
8
9 # print the number of variants
10 print("The number of variants is", len(variants))
11
```

The interface includes a top menu bar with options like File, Edit, Code, View, Plots, Session, Build, Debug, Profile, Tools, and Help. Below the menu is a toolbar with icons for file operations and a search bar. The right sidebar contains tabs for Environment, History, Connections, Git, and Tutorial. The Environment tab is active, showing a table of files:

Staged	Status	Path
☐	?	.gitignore
☐	?	myproject2.Rproj

Below the Environment tab is a file explorer showing the directory structure:

- Home > myproject2 > Src
 - ..
 - MyScript.py (147 B, Sep 27, 2023, 11:25 AM)

The bottom panel contains a console window with the R prompt and output:

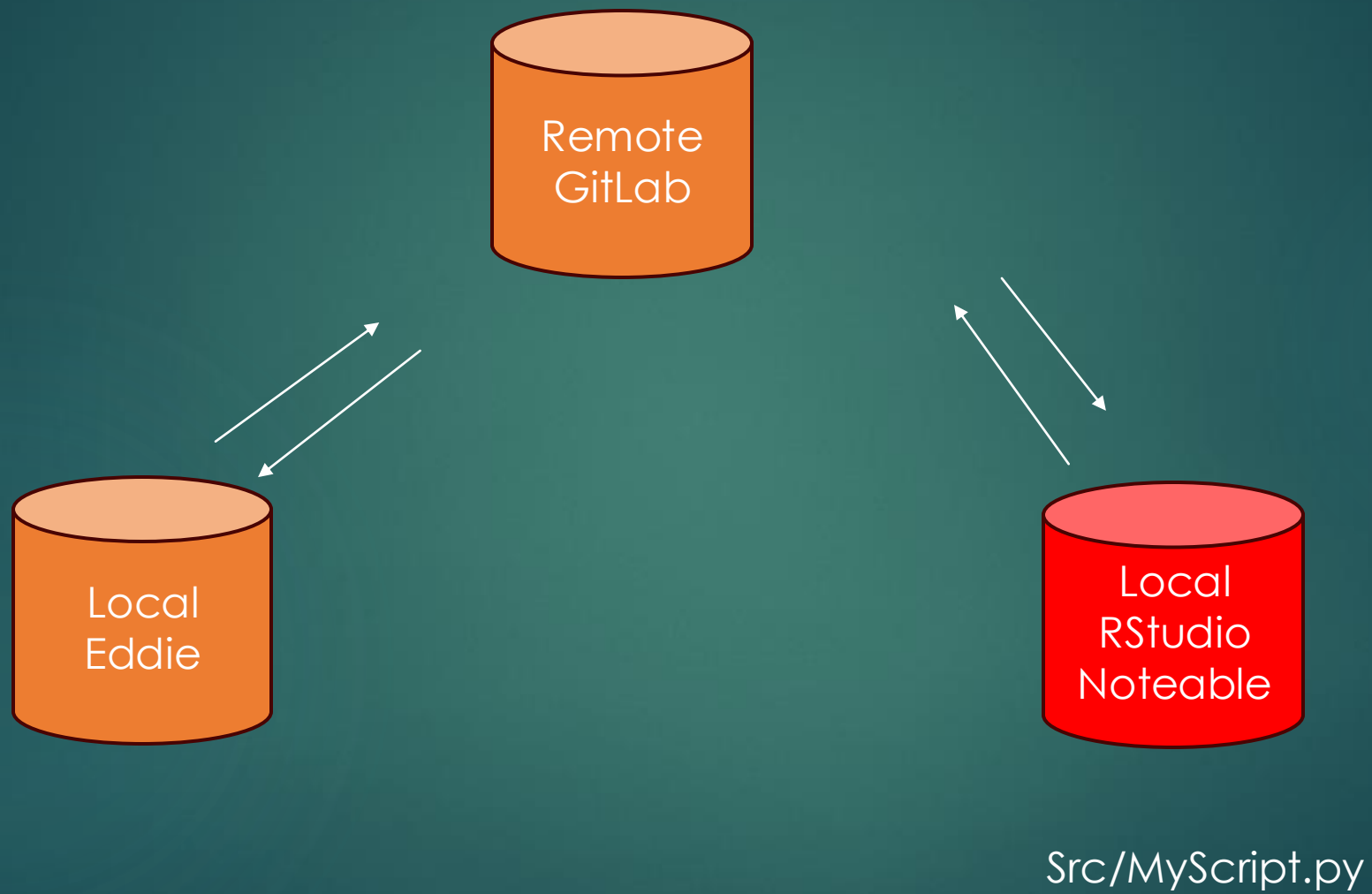
```
R 4.1.3 · ~/myproject2/

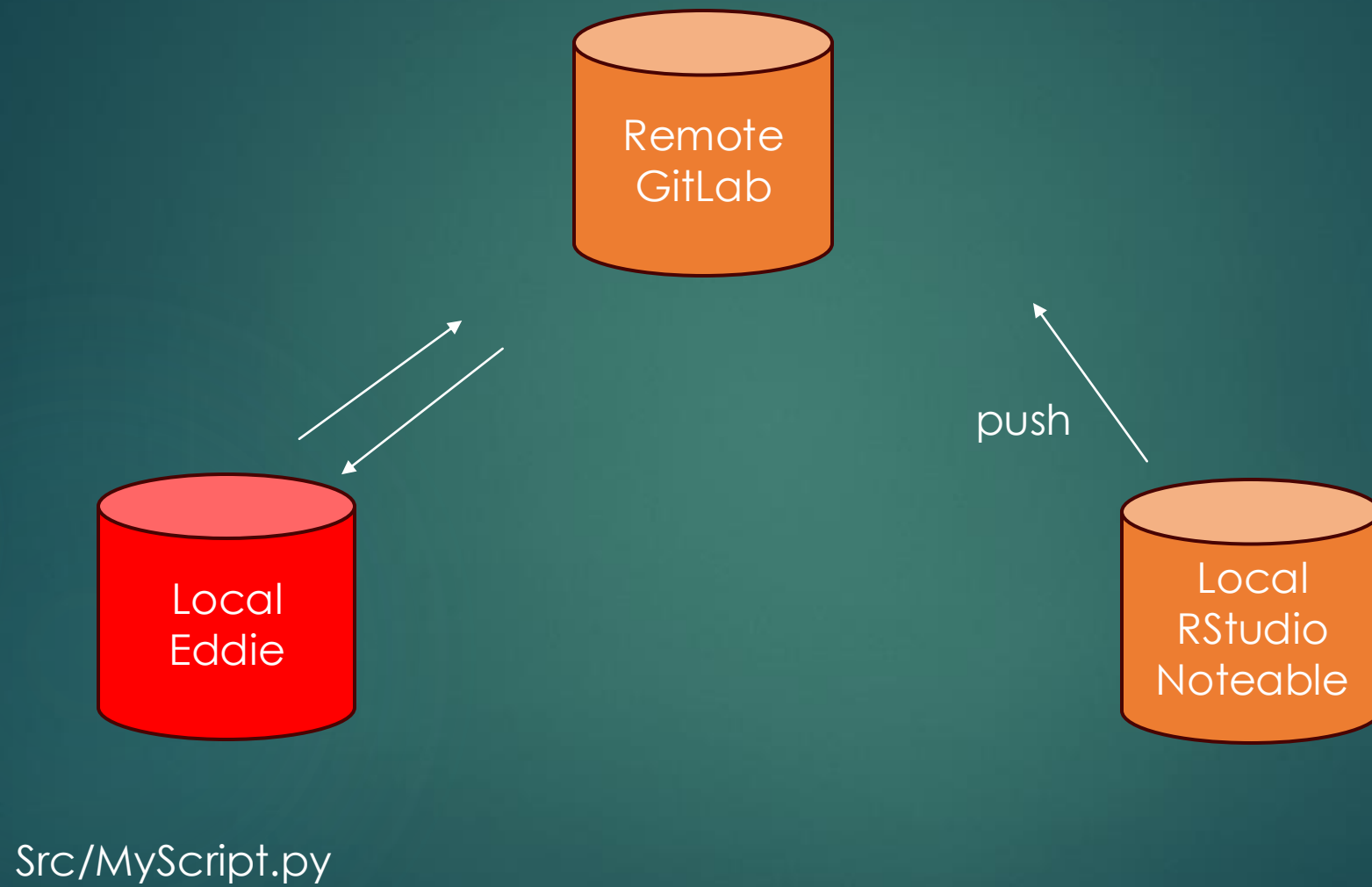
Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```





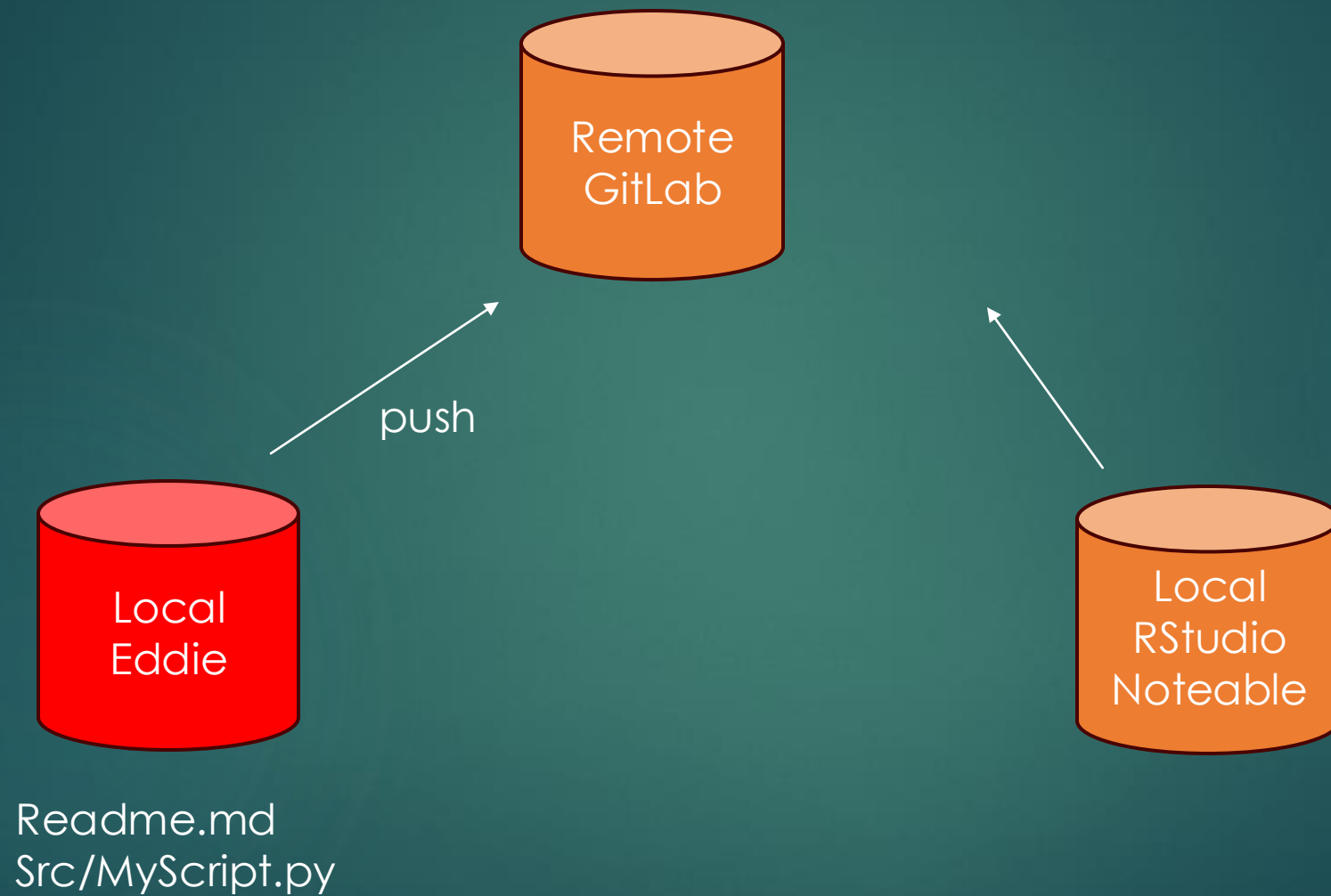
On Eddie: Change Readme.md

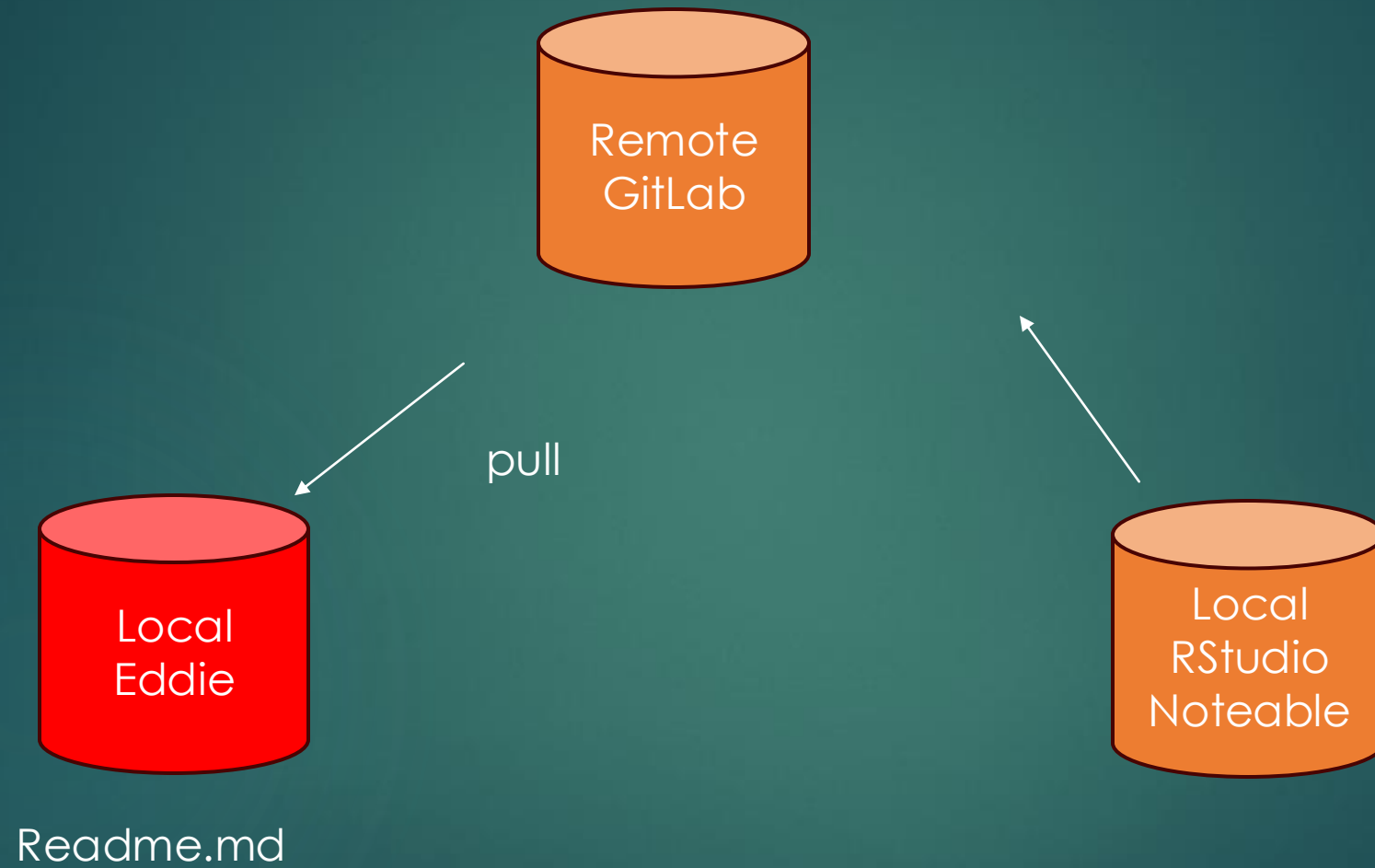
Using noteable

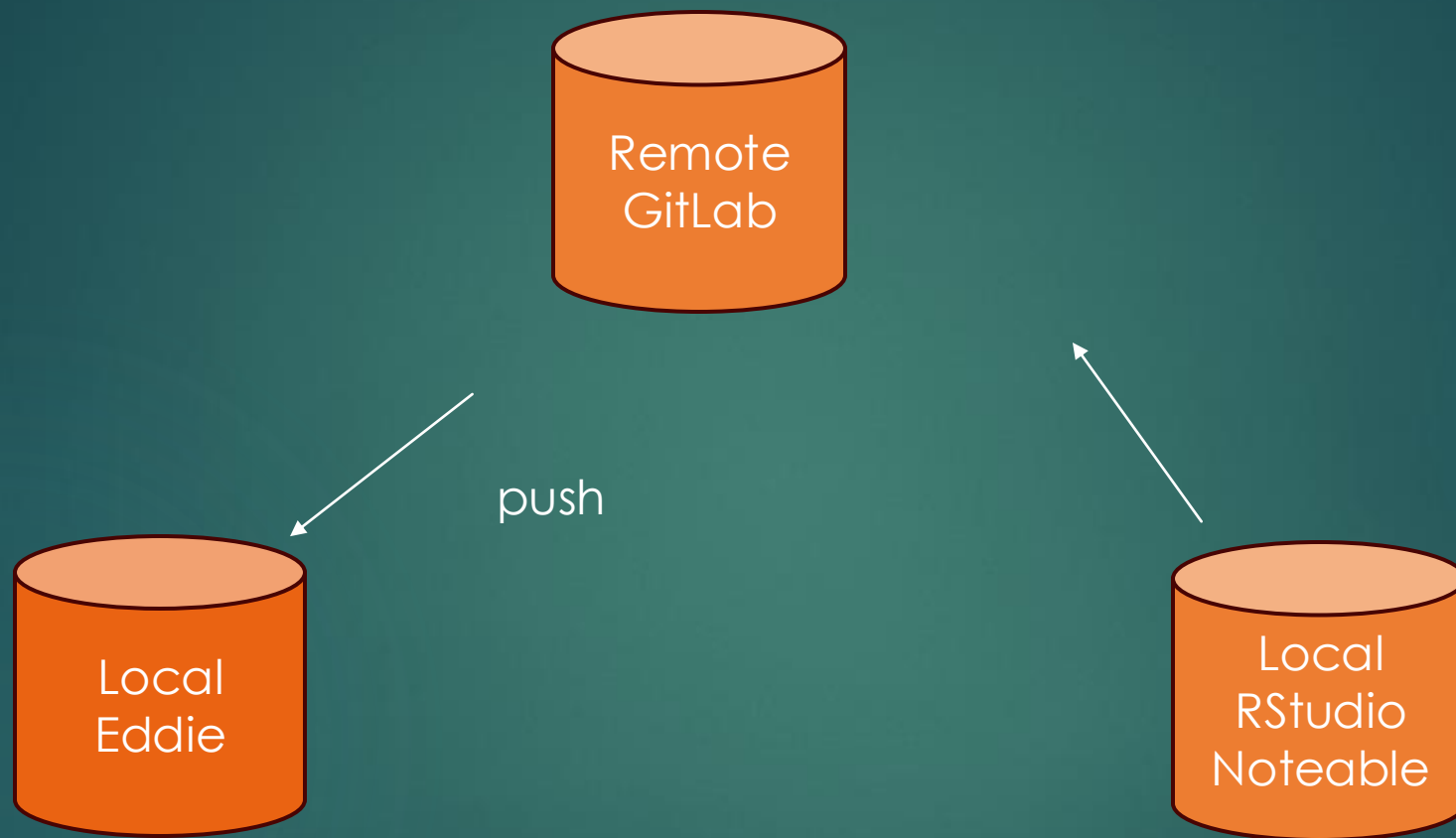
To use notable you must login to:

*

noteable.edina.ac.uk/login







Collaborating

BREAK INTO PAIRS

Different models for Collaboration

- ▶ Give people access to your repository (you need to trust the person)
- ▶ Fork a repository - produces a remote clone of the other person's repository (you can't always do this , e.g. from a private to a private repository).
- ▶ Here we are going to assume a trust model.
- ▶ We are going to pair you up.
 - ▶ Introduce yourselves.
 - ▶ Person 1 get the UUN of Person 2.

Person 1: Add a Person2 as a member to Repo

←

→

↺

git.ecdf.ed.ac.uk/ggrimes2/myproject/-/project_members

🔍

☆

👤

📧

🗨

📅

📧

📁

📁

👤

⋮

🐙

📁 + 🌐

📄 22

🔗

📧 17

🔍 Search or go to...

Project

M MyProject

📌 Pinned

📄 Issues 0

🔄 Merge requests 0

👤 Members

👤 Manage

📅 Plan

📄 Code

🛡 Secure

🚀 Deploy

🏠 Operate

📺 Monitor

📊 Analyze

⚙ Settings

Graeme Grimes / MyProject / Members

Project members

Import from a project

Invite a group

Invite members

You can invite a new member to **MyProject** or invite another group.

Members 1

🕒

Filter members

🔍

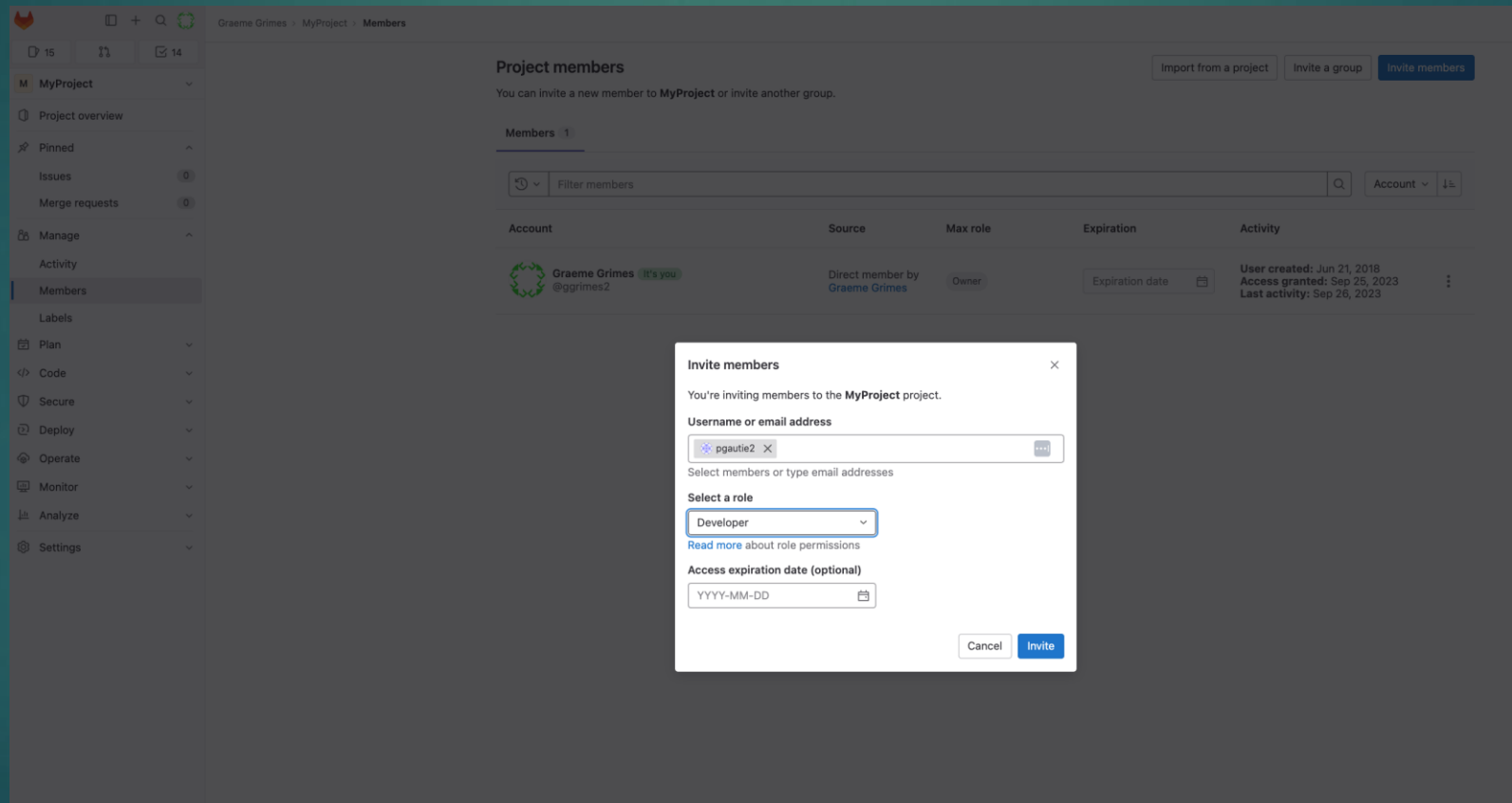
Account

⌵

⌴

Account	Source	Max role	Expiration	Activity
🌐 Graeme Grimes It's you @ggrimes2	Direct member by Graeme Grimes	Owner	Expiration date	User created: Jun 21, 2018 Access granted: Jan 18, 2024 Last activity: Jan 18, 2024

Send an Invite: Use UUN of person 2 Set as a Developer



The screenshot shows the GitHub interface for a project named 'MyProject'. The left sidebar contains navigation links: 15 repositories, 14 issues, Project overview, Pinned, Merge requests, Manage, Activity, Members (selected), Labels, Plan, Code, Secure, Deploy, Operate, Monitor, Analyze, and Settings. The main content area is titled 'Project members' and includes buttons for 'Import from a project', 'Invite a group', and 'Invite members'. Below this is a table of current members. One member, 'Graeme Grimes' (username @ggrimes2), is listed as a 'Owner' with an expiration date. An 'Invite members' modal is open in the foreground, showing the process of adding a new member. The modal includes a search bar with 'pgautie2' entered, a role selection dropdown set to 'Developer', and an optional access expiration date field. The modal also contains a 'Cancel' button and an 'Invite' button.

Project members

You can invite a new member to **MyProject** or invite another group.

Members 1

Account	Source	Max role	Expiration	Activity
 Graeme Grimes @ggrimes2 It's you	Direct member by Graeme Grimes	Owner	Expiration date 	User created: Jun 21, 2018 Access granted: Sep 25, 2023 Last activity: Sep 26, 2023

Invite members

You're inviting members to the **MyProject** project.

Username or email address

 pgautie2 

Select members or type email addresses

Select a role

Developer

[Read more](#) about role permissions

Access expiration date (optional)

YYYY-MM-DD 

Cancel Invite

GitLab Protected Branches

- ▶ In GitLab, by default , there are restrictions around who can push changes.
- ▶ This is to prevent unwanted changes.
- ▶ You need to change setting if you want a collaborator to push changes into your repository

Settings->Repository->Protected branches

The screenshot shows the GitHub repository settings page for 'MyProject' at 'git.ecdf.ed.ac.uk/ggrimes2/myproject'. The left sidebar contains navigation links: Project overview, Pinned, Issues (0), Merge requests (0), Manage, Plan, Code, Secure, Deploy, Operate, Monitor, Analyze, Settings (expanded), General, Integrations, Webhooks, Access Tokens, Repository (selected), Merge requests, Packages and registries, Monitor, and Usage Quotas. The main content area is titled 'Protected branches' and includes a 'Collapse' button. It contains a 'Protect a branch' form and a table of protected branches.

Mirroring repositories Expand

Set up your project to automatically push and/or pull changes to/from another repository. Branches, tags, and commits will be synced automatically. [How do I mirror repositories?](#)

Protected branches Collapse

Keep stable branches secure and force developers to use merge requests. [What are protected branches?](#)

By default, protected branches restrict who can modify the branch. [Learn more.](#)

Protect a branch

Branch:
main
[Wildcards](#) such as `**stable` or `production/*` are supported.

Allowed to merge:
Developers + M... ▼

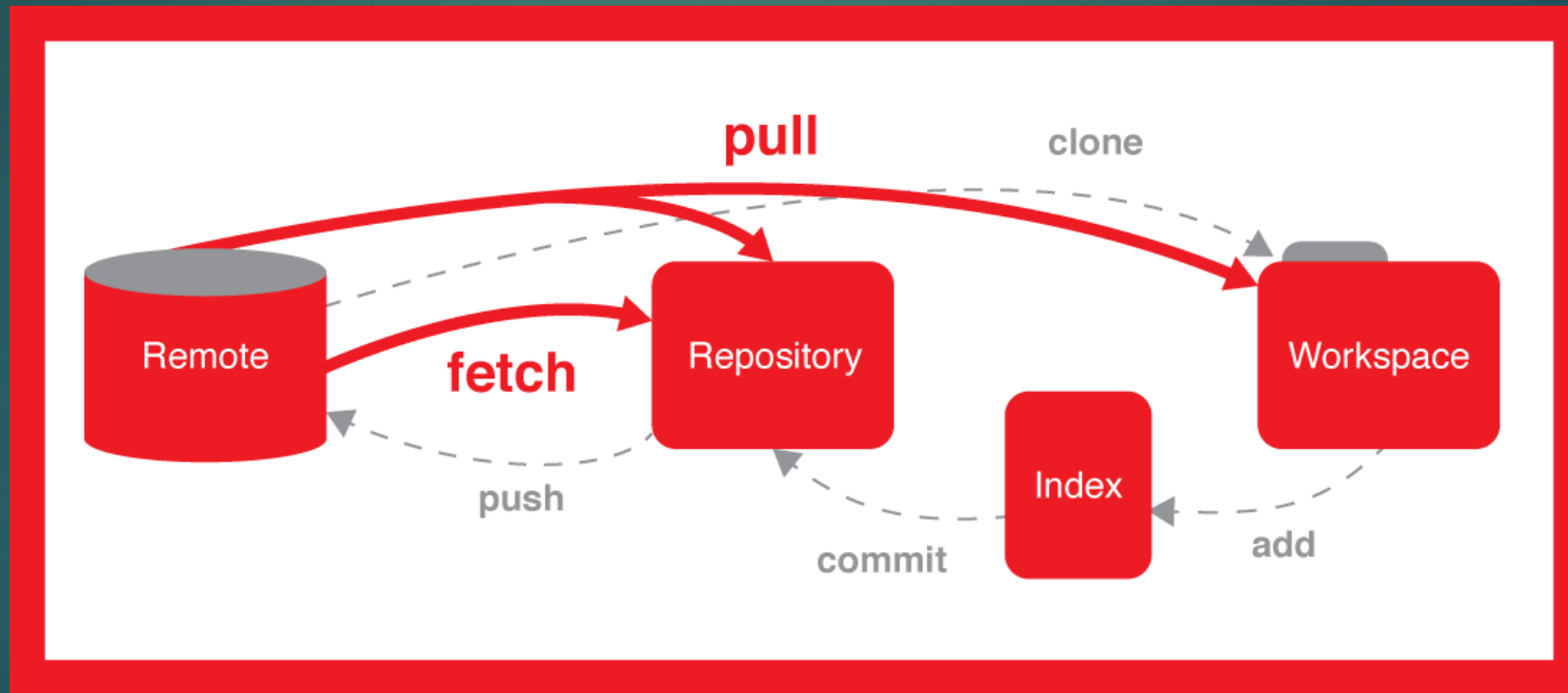
Allowed to push and merge:
Developers + M... ▼

Allowed to force push:
☒ ×
Allow all users with push access to [force push](#).

Protect

Branch	Allowed to merge	Allowed to push and merge	Allowed to force push ?
main default	Maintainers ▼	Maintainers ▼	☒ × Unprotect

Person 2 Clones Person 1 Repo on eddie



git.ecdf.ed.ac.uk/ggrimes2/myproject

MyProject

Project ID: 11548

5 Commits 1 Branch 0 Tags 0 B Project Storage

Merge branch 'main' of git.ecdf.ed.ac.uk:ggrimes2/myproject
Graeme Grimes authored 18 hours ago

ff8da650

main myproject +

README Add LICENSE Add CHANGELOG Add CONTRIBUTING Add Wiki Configure Integrations

Name	Last commit
Data	added data.csv
Src	Reading in data
Readme.md	Update Readme.md

Readme.md

My eddie Python Project

To run Python on eddie need to run:

```
module load igmm/apps/python/3.7.3
```

Data

The data is made available under an [Creative Commons license](#).

Clone with SSH
git@git.ecdf.ed.ac.uk:ggrimes2/i

Clone with HTTPS
https://git.ecdf.ed.ac.uk/ggrimes2/i

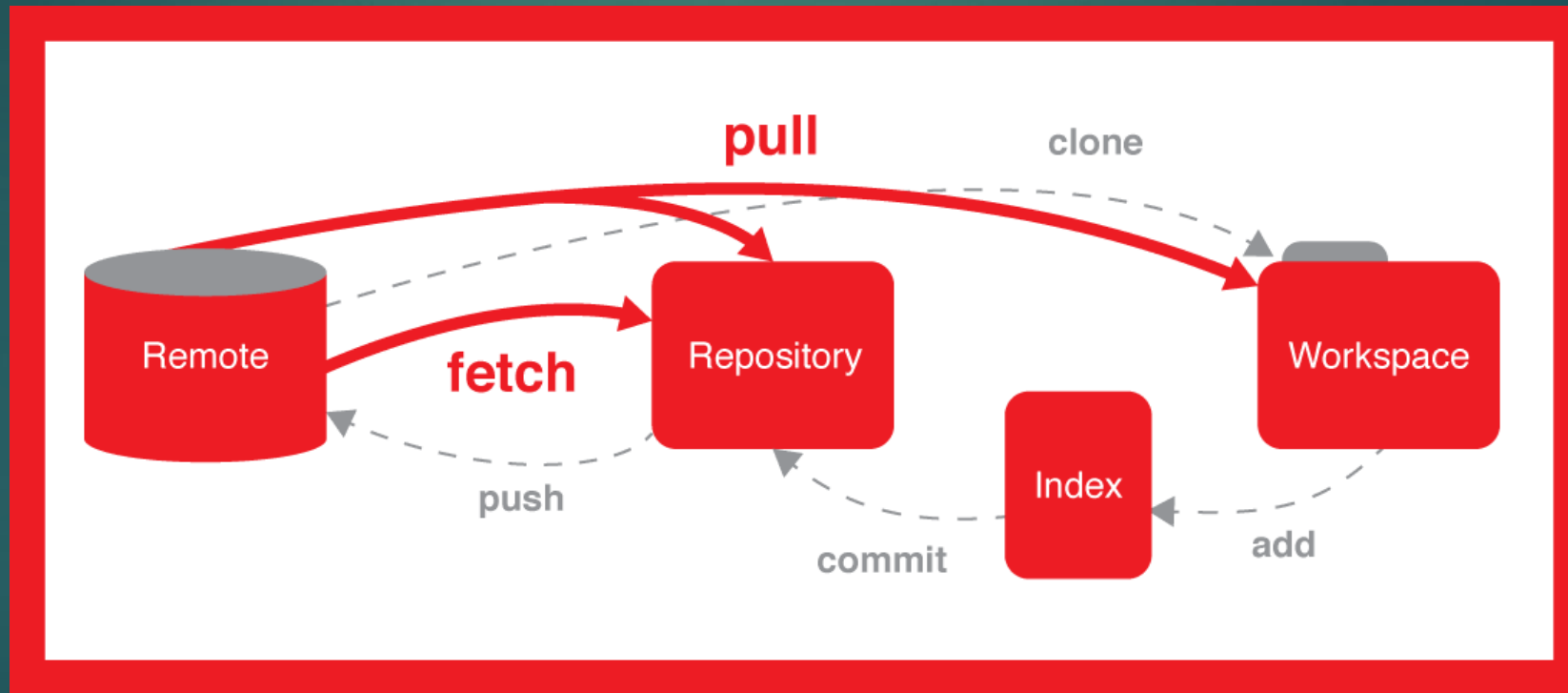
Open in your IDE
Visual Studio Code (SSH)
Visual Studio Code (HTTPS)
IntelliJ IDEA (SSH)
IntelliJ IDEA (HTTPS)

```
2023.03[ggrimes2@node2c17(eddie) ggrimes2]$ git clone https://git.ecdf.ed.ac.uk/ggrimes2/myproject.git project-collab
Cloning into 'project-collab'...
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 25 (delta 2), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (25/25), 23.01 KiB | 4.60 MiB/s, done.
Resolving deltas: 100% (2/2), done.
```

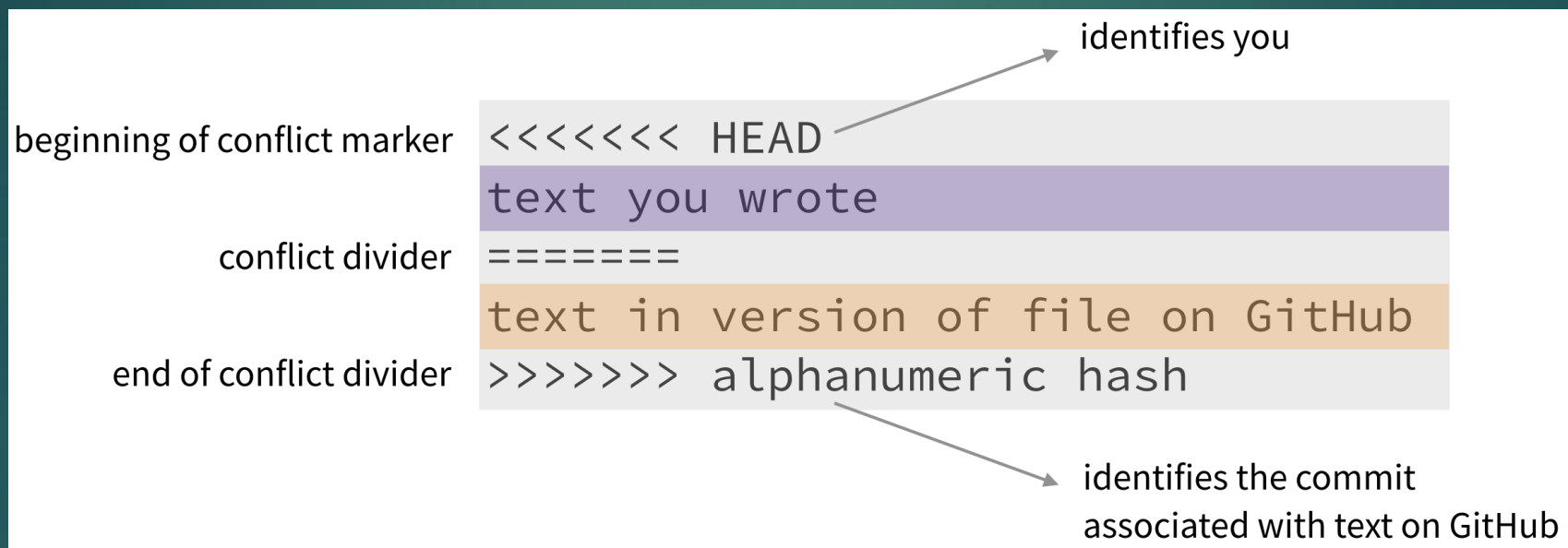
Person 2 : git blame

```
ggrimes2@node2c17(eddie) project-collab]$ git blame Readme.md
(Graeme Grimes 2023-09-25 16:05:37 +0100 1) # My eddie Python Project
(Graeme Grimes 2023-09-25 16:11:26 +0100 2)
(Graeme Grimes 2023-09-25 16:11:26 +0100 3) To run Python on eddie need to run:
(Graeme Grimes 2023-09-25 16:11:26 +0100 4)
(Graeme Grimes 2023-09-25 16:11:26 +0100 5) module load igmm/apps/python/3.7.3
(Graeme Grimes 2023-09-25 16:31:02 +0100 6)
(Graeme Grimes 2023-09-25 16:31:02 +0100 7) ## Data
(Graeme Grimes 2023-09-25 16:31:02 +0100 8)
(Graeme Grimes 2023-09-25 16:31:02 +0100 9) The [data](https://figshare.com/articles/dataset/Data_Carpentry_Genomic
0/7726454?file=14632895) is made available under an [Creative Commons license](https://creativecommons.org/licenses
).
```

add -> commit -> push
(if you get rejected you need to do a pull first).



Merge conflicts



Caching Credentials

- ▶ Cache Credentials: Useful if you're tired of entering your credentials all the time.
- ▶ # Cache for 1 hour (3600 seconds)

```
$ git config --global credential.helper 'cache --timeout=3600'
```

Fork: Another way of collaborating

<https://git.ecdf.ed.ac.uk/ggrimes2/git4igc>

Click on the **Fork** button. This will do **a remote copy of my repo** and put it up as a remote copy in your GitLab space.

The screenshot displays the GitLab web interface. The top navigation bar shows the project path `git.ecdf.ed.ac.uk/ggrimes2/git4igc`. The main content area shows the project overview for **Git4IGC** (Project ID: 10772), including statistics like 59 commits, 2 branches, and 0 tags. A 'Fork' button is visible in the top right of the project overview. Below this, the 'Fork project' dialog is open, showing the 'Forked from' information and a 'Fork project' button. The dialog also includes a 'Project name' field (containing 'git4igc'), a 'Project URL' field (containing 'https://git.ecdf.ed.ac.uk/'), and a 'Project slug' field (containing 'git4igc'). The 'Project description (optional)' field contains 'Git course for the Institute of Genetics and Cancer'. The 'Visibility level' is set to 'Internal'. The 'Fork project' button is highlighted in blue.



Exploring history

git log: a history of commits.

- ▶ Show the sequence of commits starting from the current one, in reverse chronological order of the Git repository in the current working directory:
 - `$ git log`
- ▶ As before but using only the first line of each commit message
 - `$ git log --oneline`

git diff: Show changes to tracked files

- ▶ We can use the git diff command to show the differences between
 - ▶ two commits
 - ▶ `$git diff [commit_ID1] [commit_ID2]`
 - ▶ between a commit and the working directory
 - ▶ `$git diff [commit_ID1]`
 - ▶ Show only staged (added, but not yet committed) changes
 - ▶ `$git diff --staged`

git diff: On a specific file

- ▶ Note A single git commit may contain multiple files.
- ▶ You can do a `git diff` with a specific a file and a specific revision to see changes made between and a previous version:
 - ▶ `$ git diff [commit_ID] Readme.md`
- ▶ It's important to note that any changes that have been staged but not yet committed will not be included in this `git diff` comparison.
- ▶ Remember `command` only considers changes in the working directory that have not yet been staged.
 - ▶ `$ git diff --staged [commit_ID] Readme.md`

Exercise

What does the `git diff HEAD` command do?

- ▶ a) Shows the differences between the working directory and the staging area.
- ▶ b) Shows the differences between the staging area and the last commit.
- ▶ c) Shows the differences between the working directory and the last commit.
- ▶ d) Shows the differences between two different commits.

git diff shortcut HEAD~N

- ▶ `git diff HEAD~N Readme.md`
- ▶ `$ git diff HEAD~2 Readme.md`

git show

- ▶ If you want the specific contents of a file at given revision you can do:
 - ▶ `$ git show RevNum:Readme.md`

git checkout/restore

- ▶ **Replace** a file in the current working directory with the version of it committed in a given branch:
 - ▶ `$ git checkout -- Readme.md`
 - ▶ `$ git checkout RevNum -- Readme.md`
- ▶ If you want to recover the state of the repository (all files and dir) you can go back to HEAD by doing:
 - ▶ `git checkout main`
- ▶ `git restore` (Requires Git version 2.23+.). Simple alias
 - ▶ `$ git restore Readme.md`

Ignoring Things

Ignoring Things

- ▶ `$ mkdir results`
- ▶ `$ touch a.dat b.dat results/a.out results/b.out`
- ▶ `$ git status`
- ▶

.gitignore

- ▶ You have seen that `git status` is a really useful command to show you what has been changed.
- ▶ However, if you have files that you do not want to put in the repository such as emacs back up files, binary files, etc you can tell git to ignore these by creating a `.gitignore` file.
- ▶ You can create this file and add content that will apply recursively:

.gitignore

- ▶ add and commit this file and you will reduce the clutter you get.

```
$ git add .gitignore
```

```
$ git commit -m "Files to ignore."
```

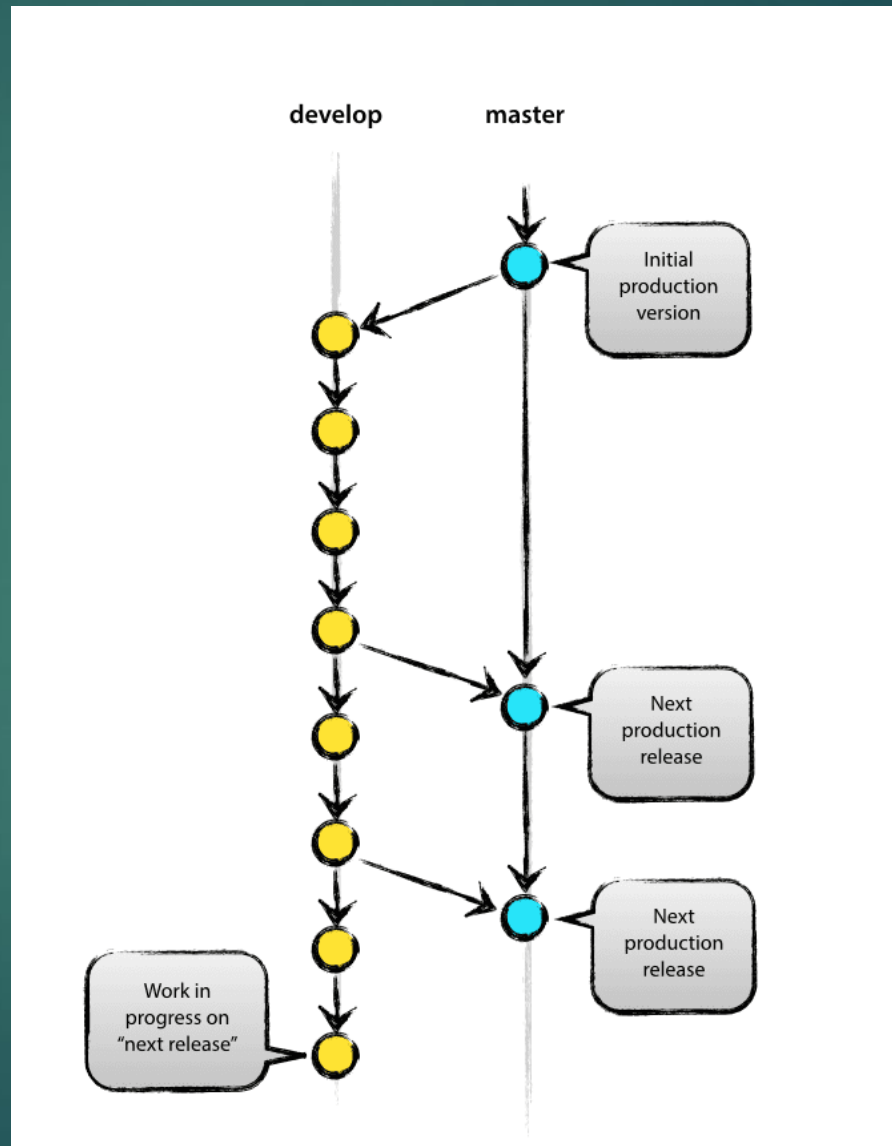
- ▶ git will not let you commit files that are contained in the .gitignore file.



Branching

PARALLEL DEVELOPMENT OF CODE

- Branching allows a very powerful way of forking your code.]
- Want to test out a change in a script.
- You are collaborating and you do not want to be modifying the same code at the same time.



git branch

- ▶ List branches
 - ▶ `$ git branch`
- ▶ Create branch dev
 - ▶ `$ git branch dev`
- ▶ Move to dev branch
 - ▶ `$ git checkout dev`
 - ▶ `$ git branch`

Add a new file to dev branch

- ▶ `nano Src/HelloWorld.py`

```
#!/bin/env python
```

```
print("Hello World!")
```

- ▶ Switch to main branch

- ▶ `$ git checkout main`

- ▶ `$ ls Src`

- ▶ Incorporate changes

- ▶ `$ git merge dev`

- ▶ Push branch dev to remote

- ▶ `$ git push origin dev`

git diff main .. dev

css

 Copy code

```
A - B - C (main)
      \
      D - E (dev)
```

- `git diff main..dev` shows the difference between `C` and `E`.
- `git diff main...dev` shows the difference between `B` (the point where `dev` branched off) and `E`.

Close

what we should now know

- ▶ Know how initialise a local repository
- ▶ How to add files
- ▶ How to commit files
- ▶ How to navigate between different versions
- ▶ How look at the file log
- ▶ How to tag files
- ▶ Know what a branch is

Additional information

- ▶ Visual Git Reference - pictorial representations of what Git commands do
 - ▶ <http://marklodato.github.io/visual-git-guide/index-en.html>
- ▶ Pro Git - the "official" online Git book
 - ▶ <http://git-scm.com/book>
- ▶ Version control by example - an acclaimed online book on version control by Eric Sink
 - ▶ <http://www.ericsink.com/vcbe/index.html>
- ▶ Schematically visualising git interactions
 - ▶ <https://git-school.github.io/visualizing-git/>

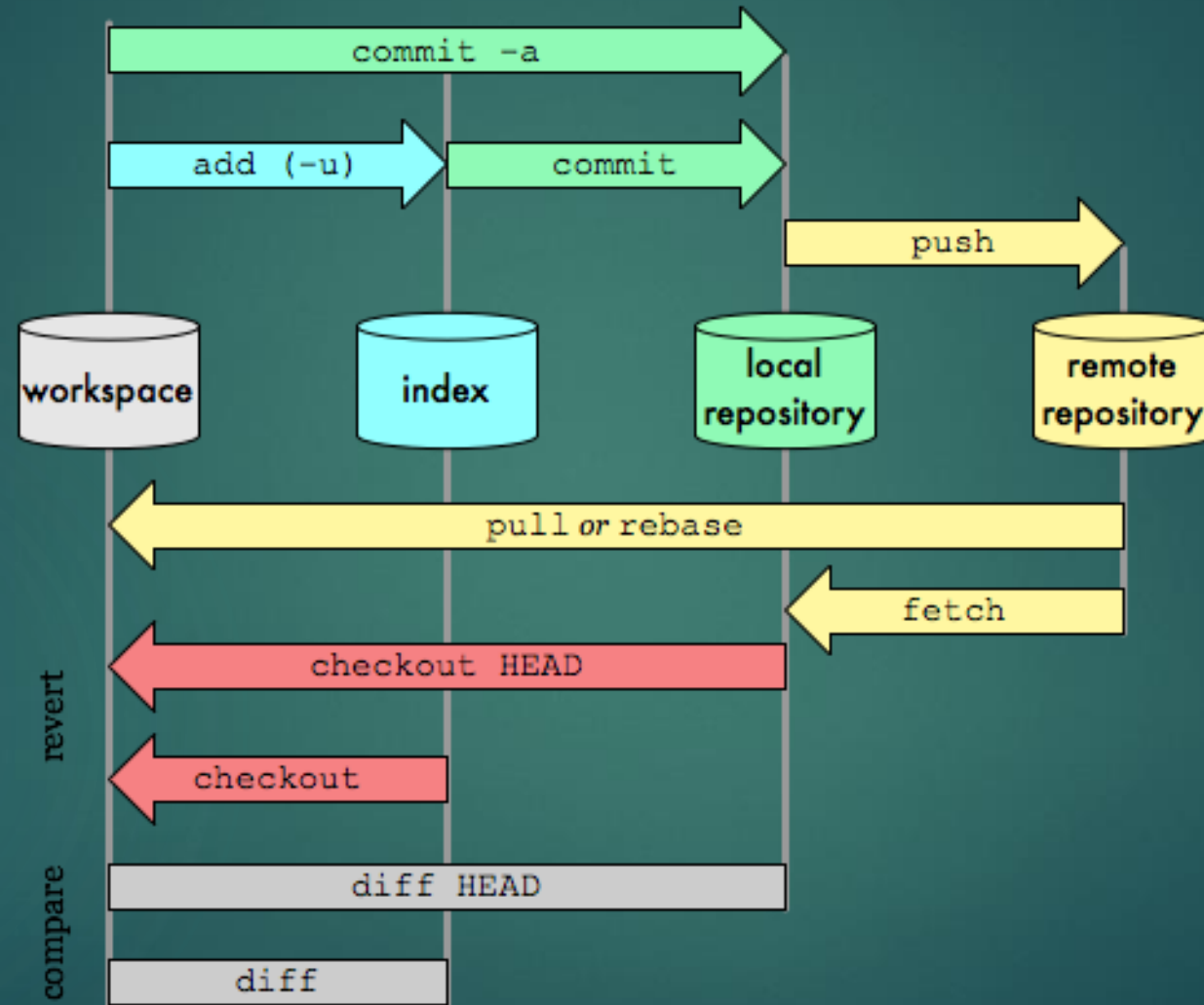
A fast-forward merge

A fast-forward merge can occur when there is a linear path from the current branch tip to the target branch. Instead of “actually” merging the branches, all Git has to do to integrate the histories is move (i.e., “fast forward”) the current branch tip up to the target branch tip.

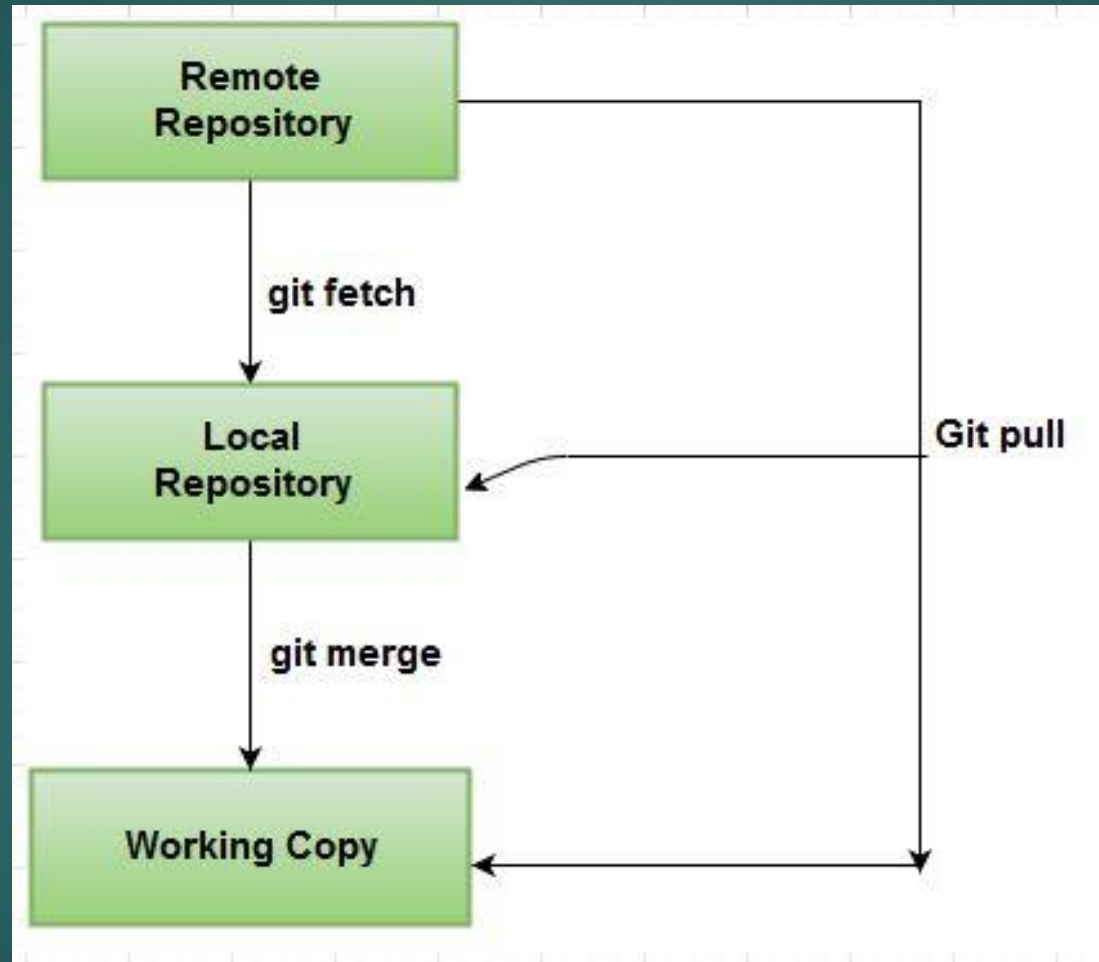
```
(base) ggrimes2@S34-YD452V3V git_diff % git log --oneline
83e878e (HEAD -> main, dev) updated README.md with letters
b68bd13 updated README.md
23bd868 added README.md
```

Git Data Transport Commands

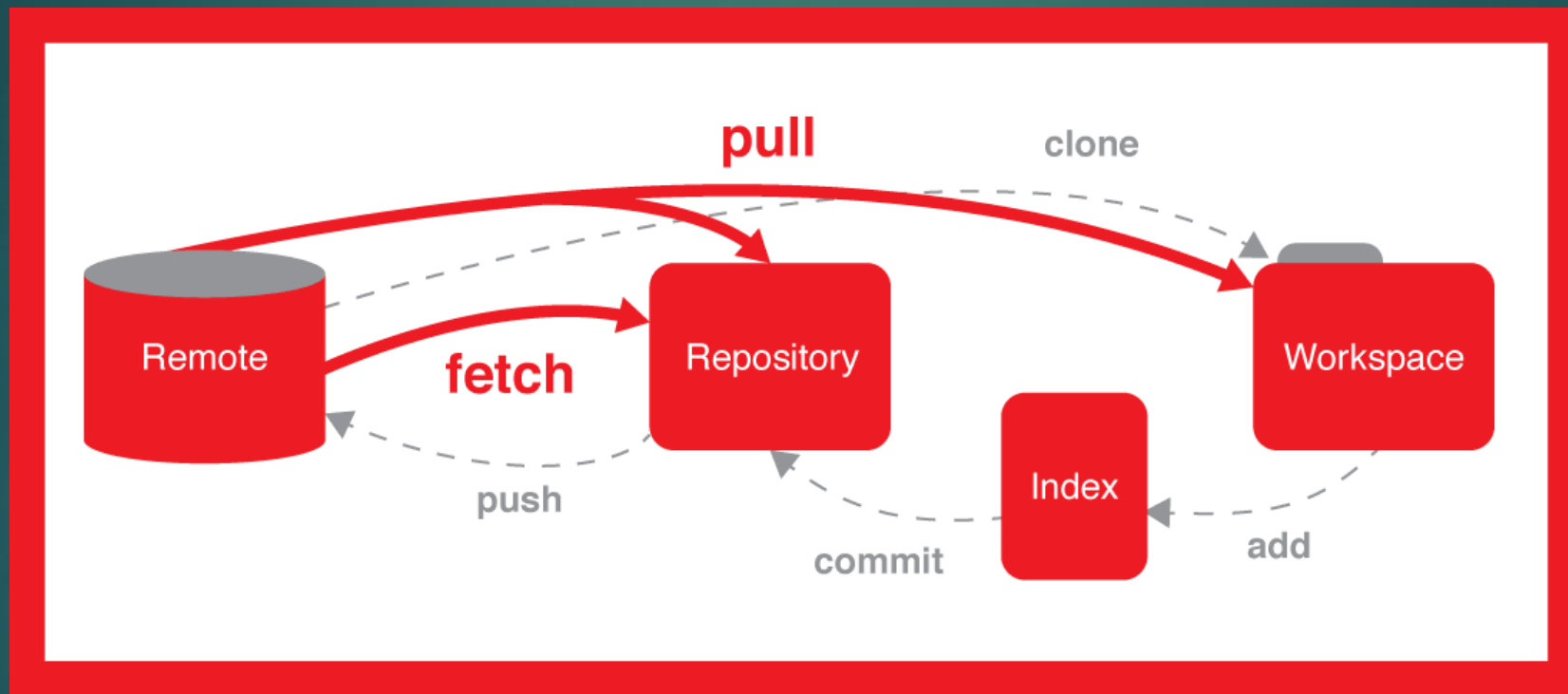
<http://osteele.com>



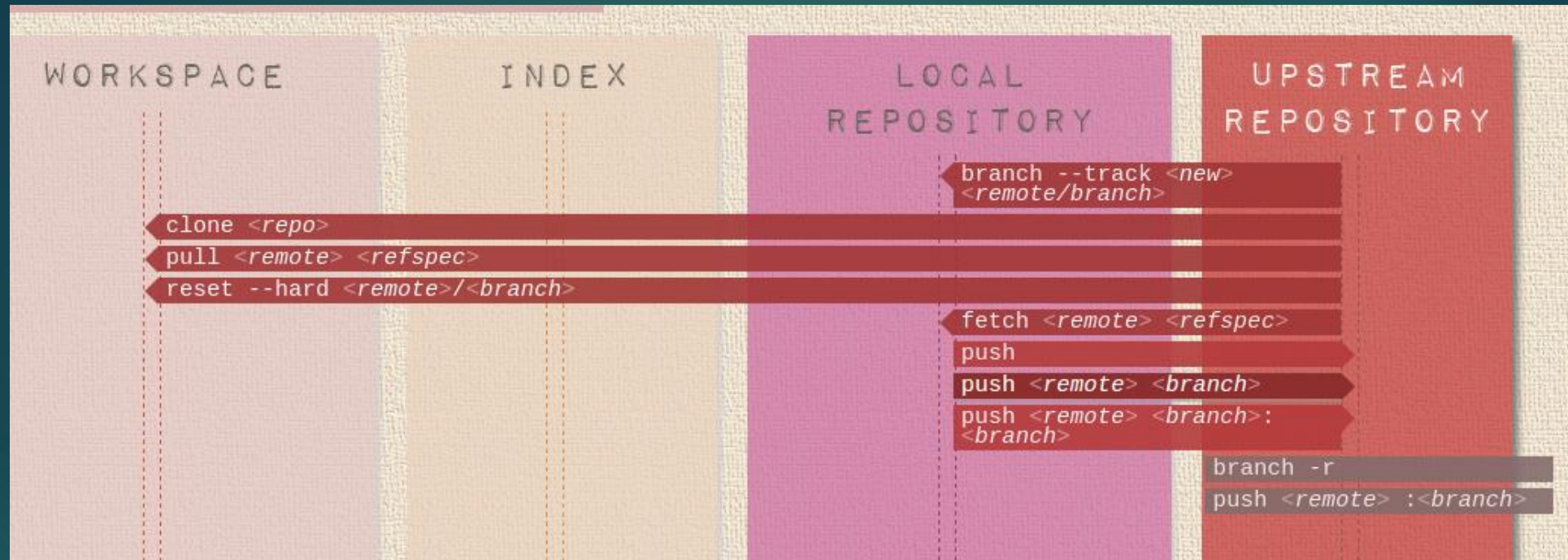
<https://stackoverflow.com/questions/292357/what-is-the-difference-between-git-pull-and-git-fetch>



<https://stackoverflow.com/questions/292357/what-is-the-difference-between-git-pull-and-git-fetch>



<https://stackoverflow.com/questions/292357/what-is-the-difference-between-git-pull-and-git-fetch>



This interactive graphical representation is very helpful in understanding git: <http://ndpsoftware.com/git-cheatsheet.html>
git fetch just "downloads" the changes from the remote to your local repository. git pull downloads the changes and merges them into your current branch. "In its default mode, git pull is shorthand for git fetch followed by git merge FETCH_HEAD."

detached head

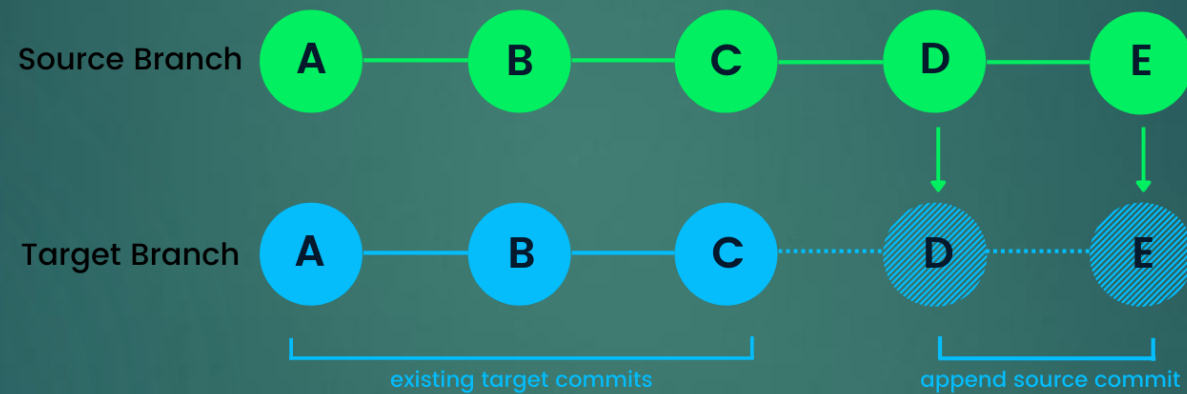


If you have ever been in a “detached head” state using Git, you might be wondering how did I get here and how do I fix this?

If you are in a ‘detached head’ state, that basically means you are no longer on any branch. You probably checked out a specific commit. To fix this problem, all you have to do is checkout the branch you were previously on:

```
$ git checkout main
```

Merge Types: Fast Forward

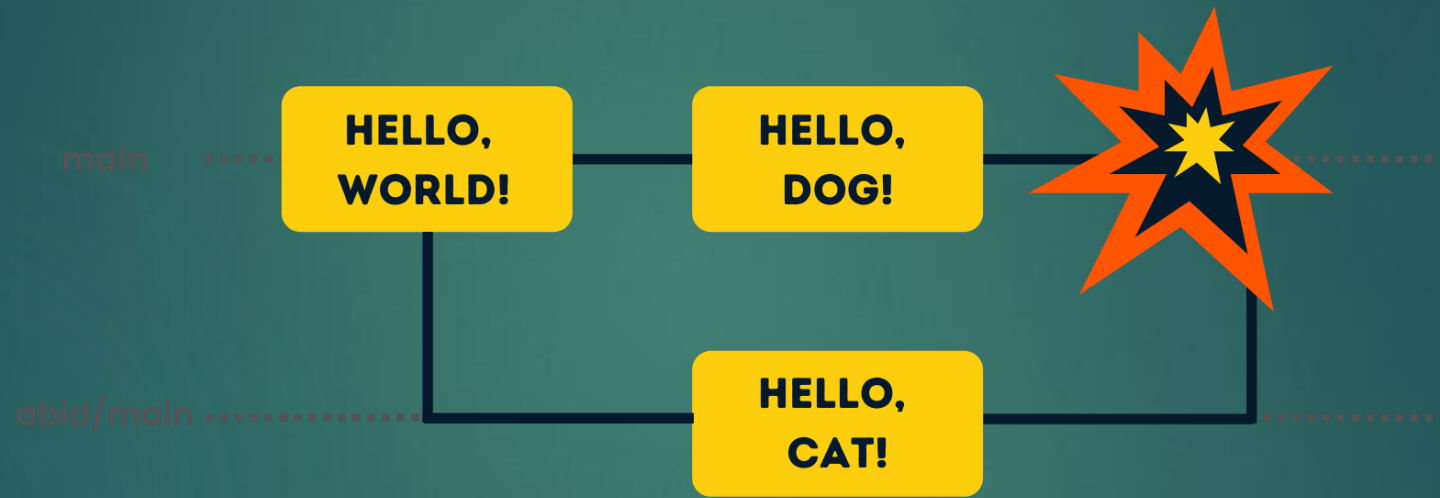


By default, Git merge uses fast-forward to integrate missing commits into the target branch. For example, it is used to update the local branch from a remote server using the pull command. Fast-forward does not raise merge conflict issues as Git will not apply it if the head of the target branch is missing in the source branch.

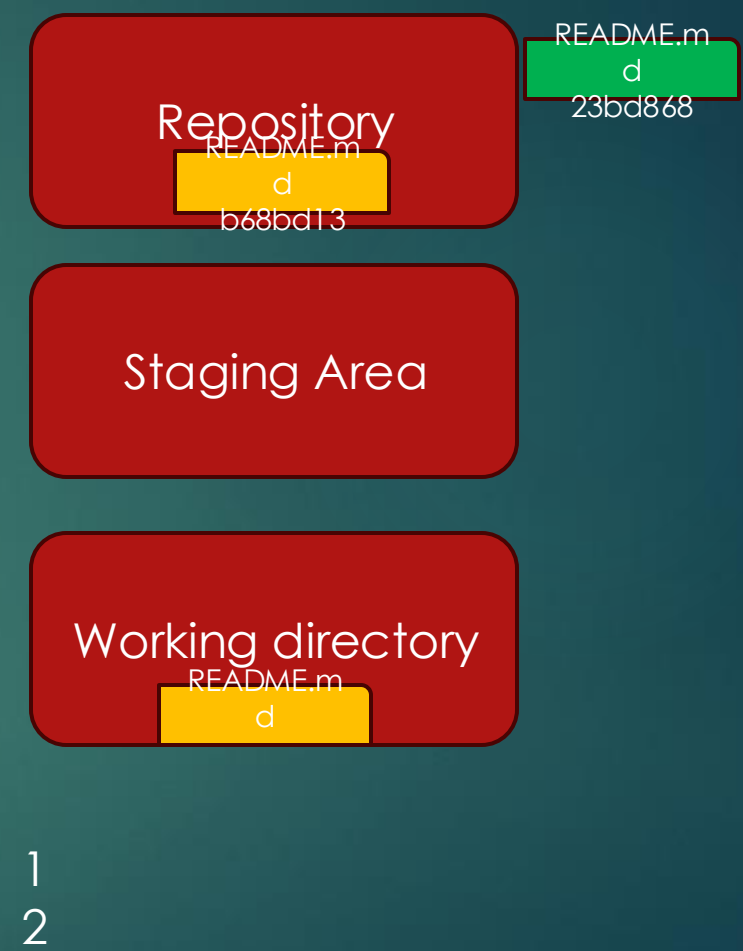
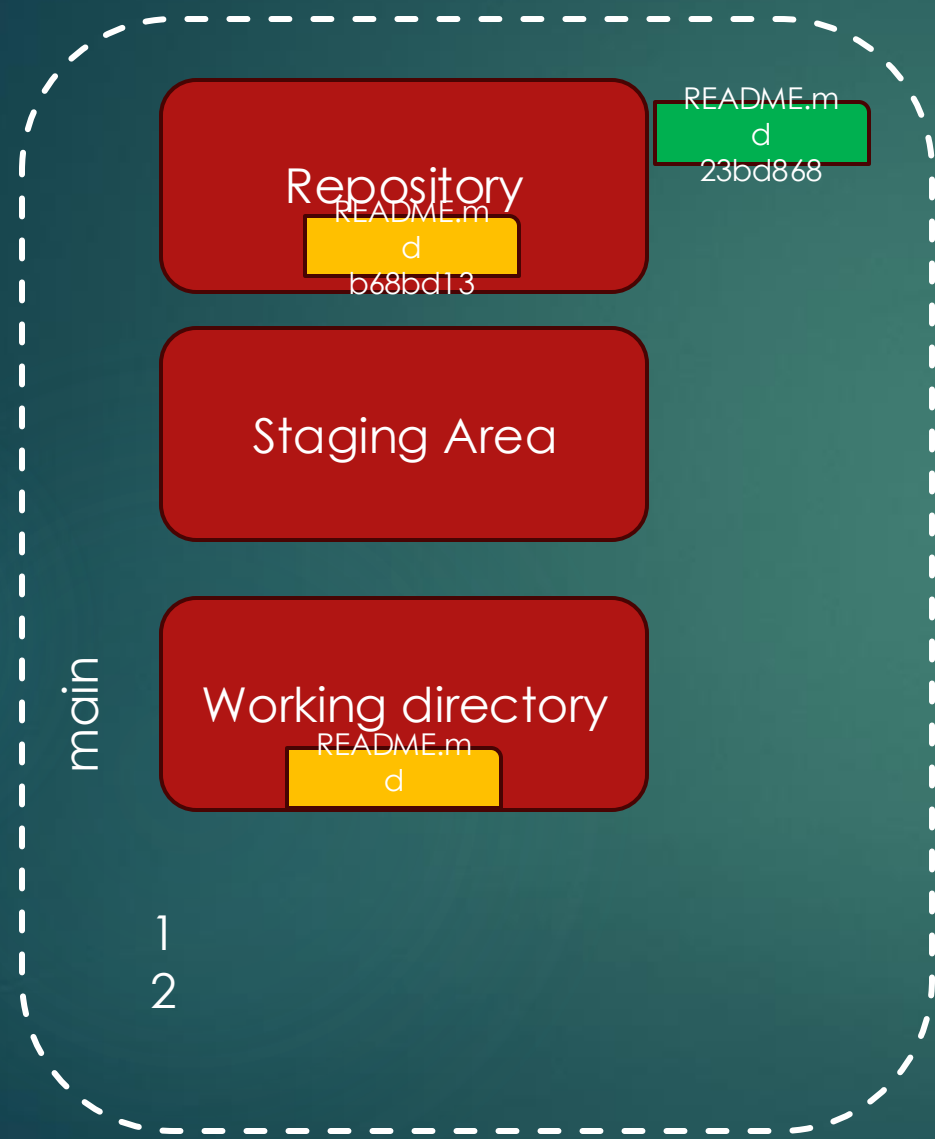
Finding Things git grep

```
$ git grep z $(git rev-list --all)
```

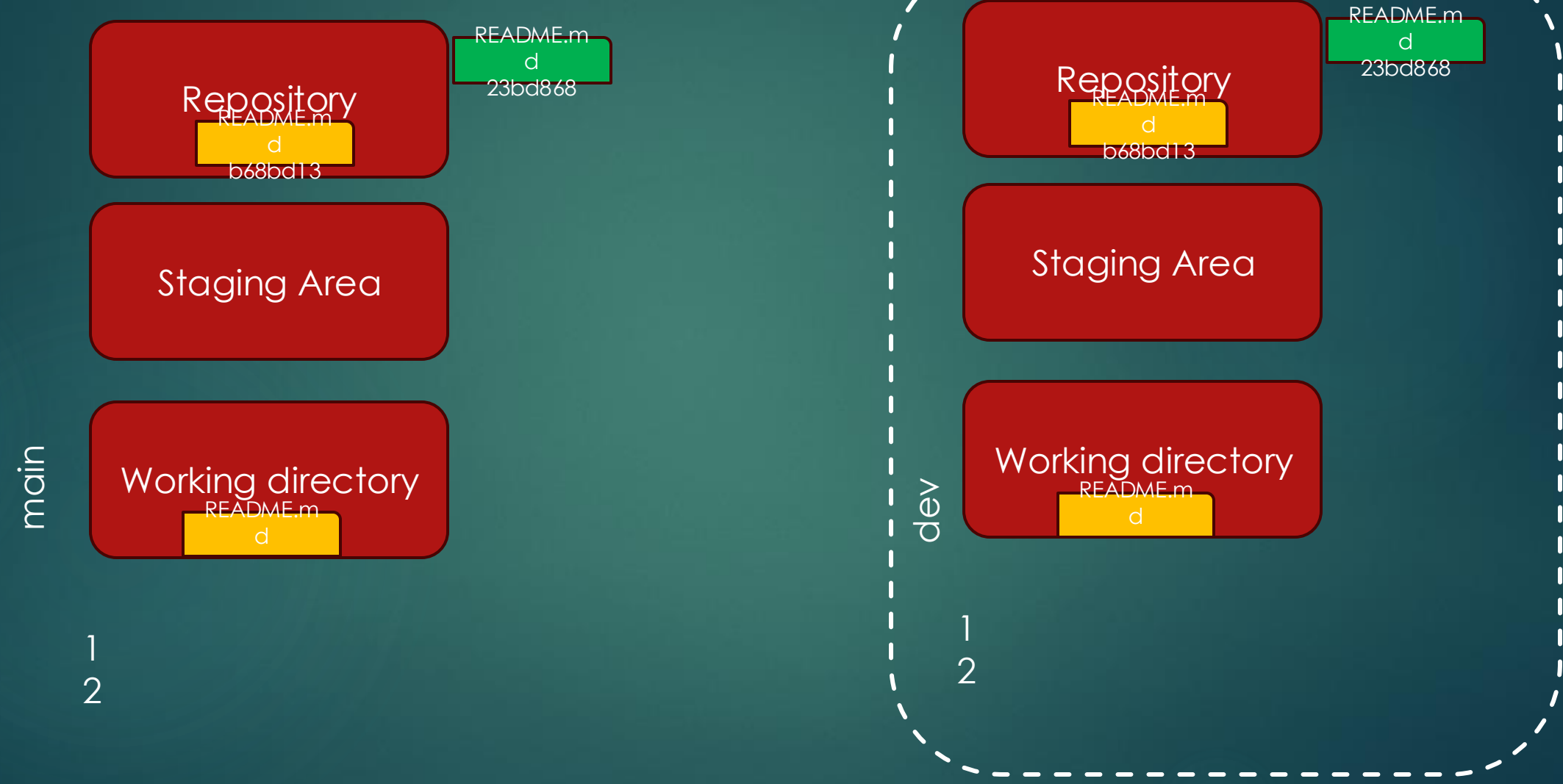
Girt Merge



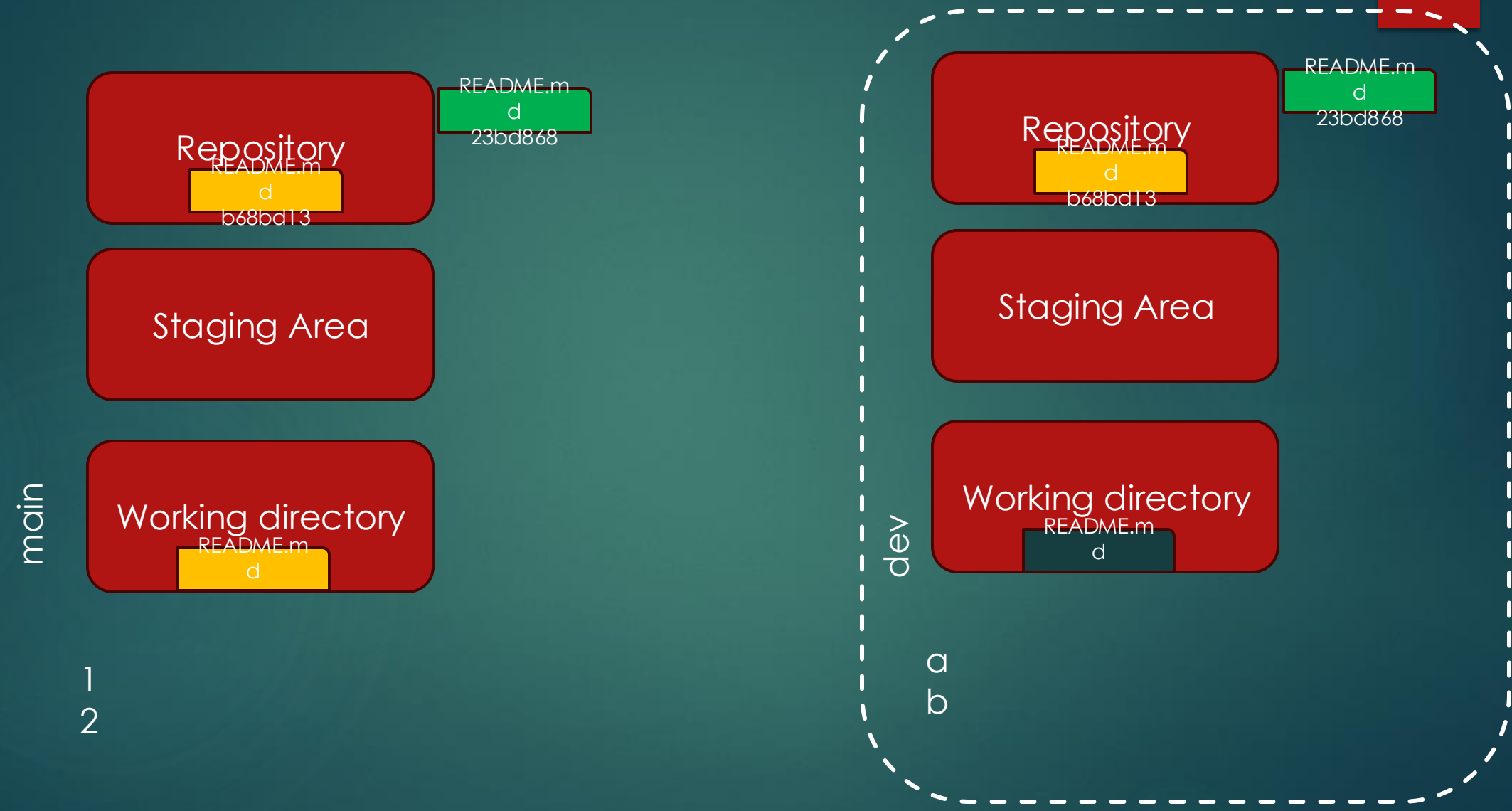
git branch dev



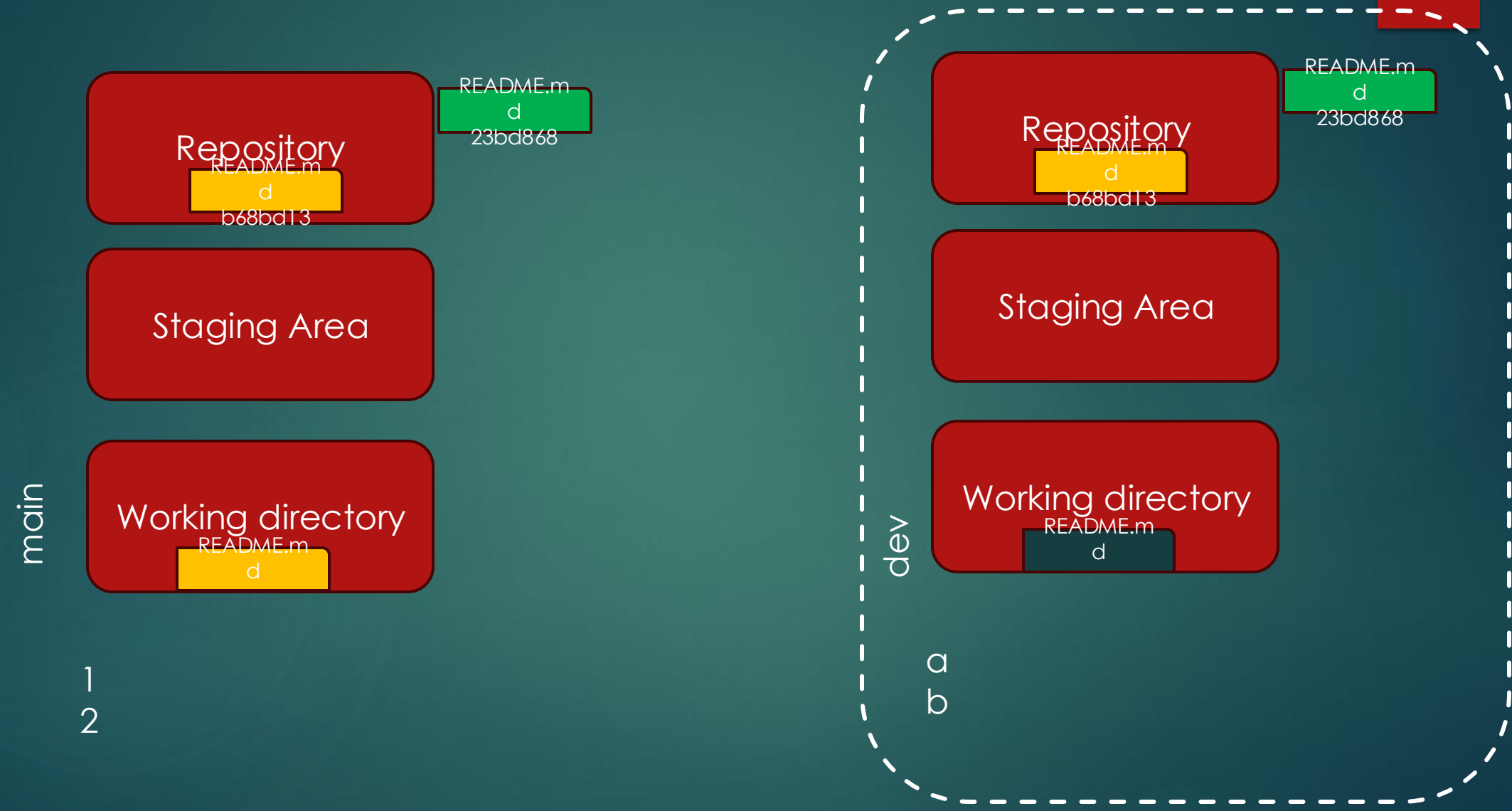
git checkout dev



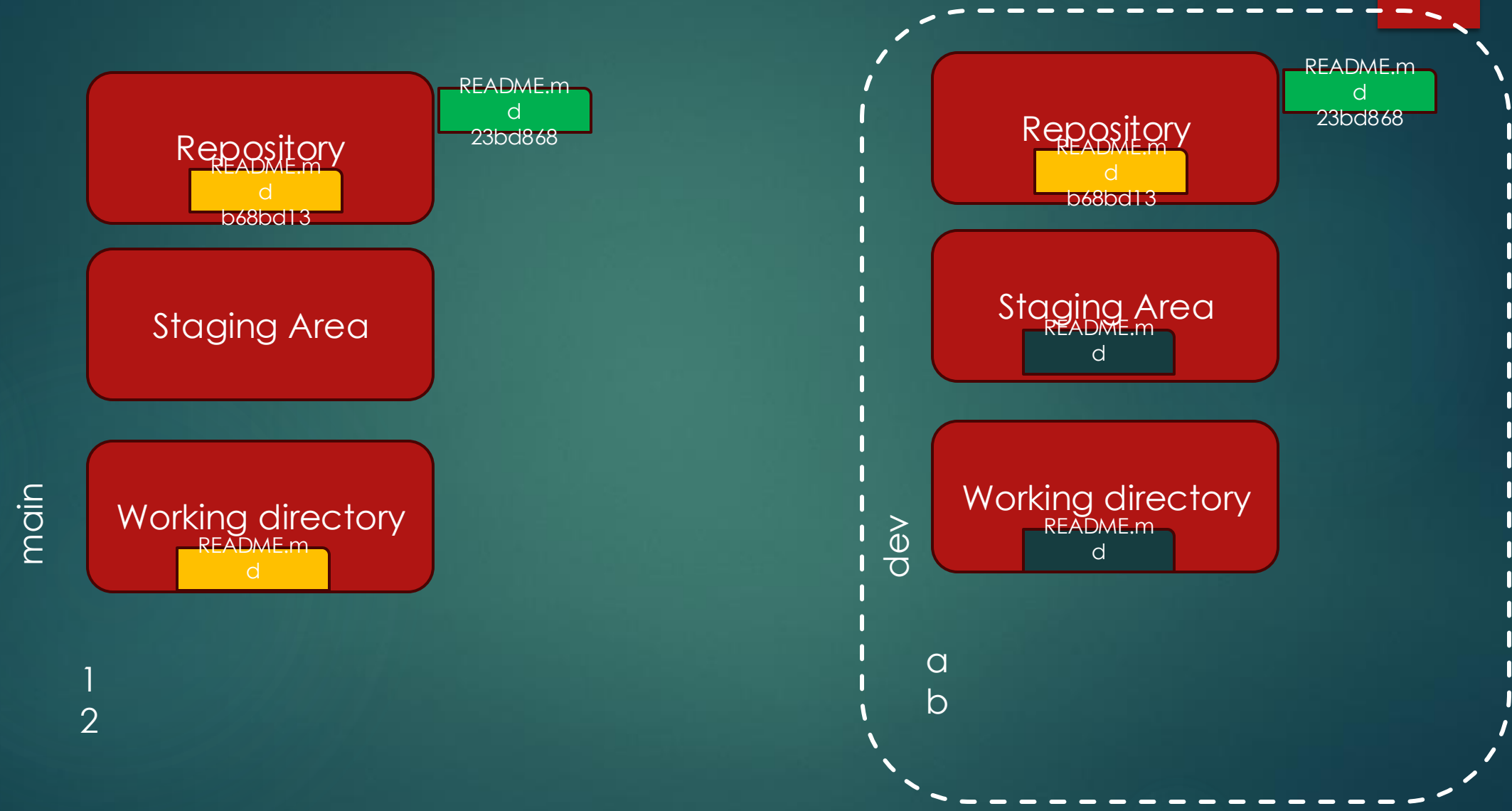
Modify README.md



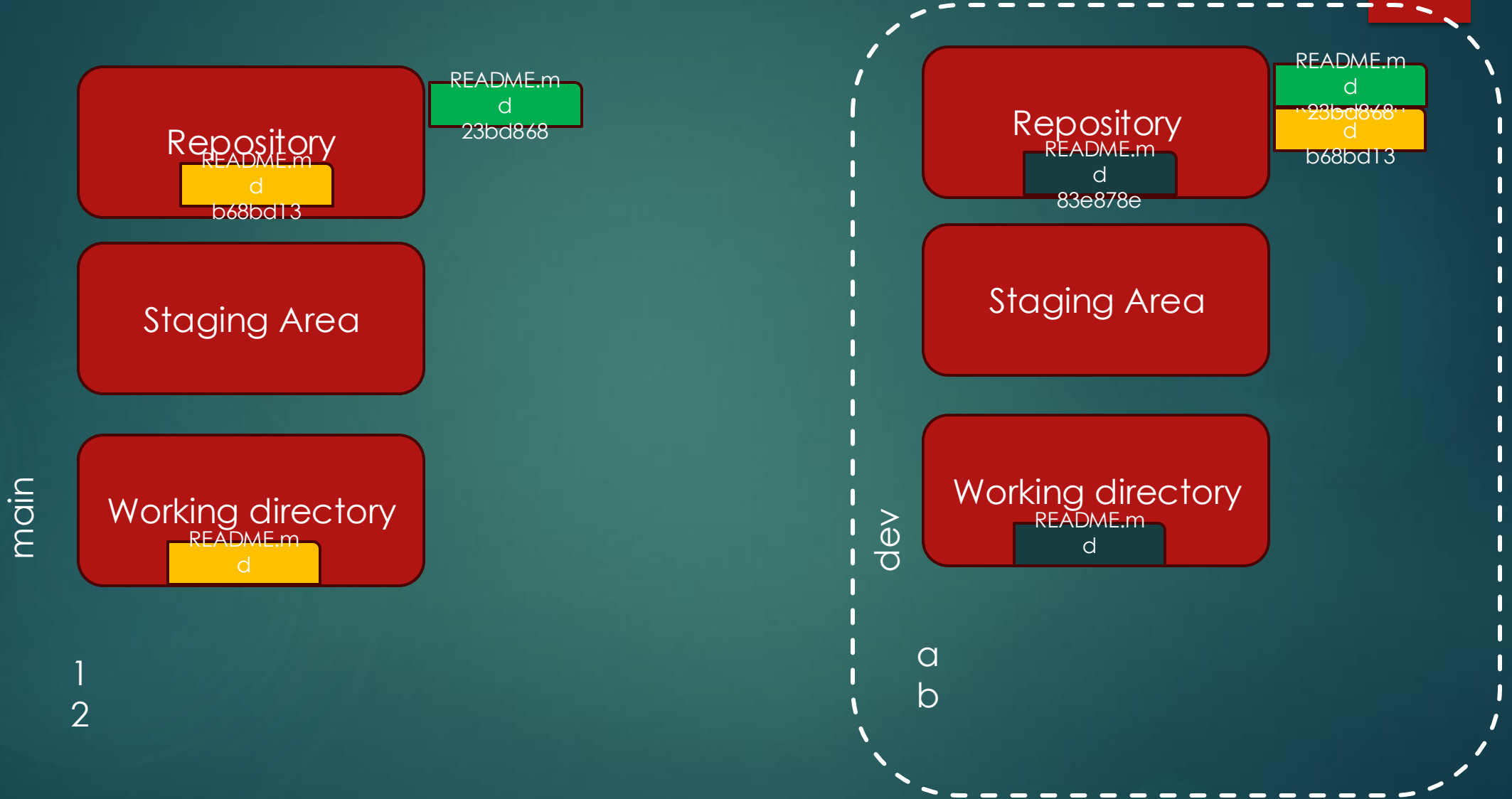
Modify README.md



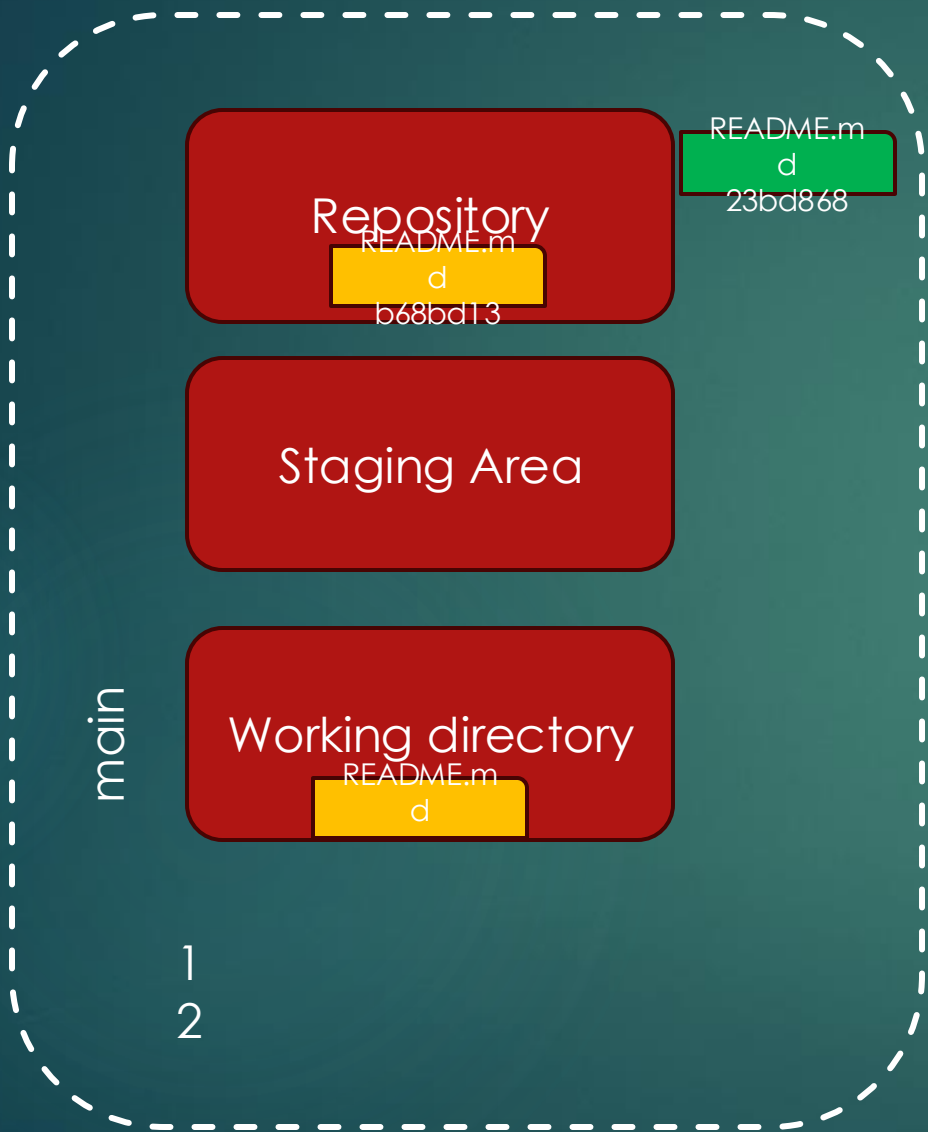
git add README.md



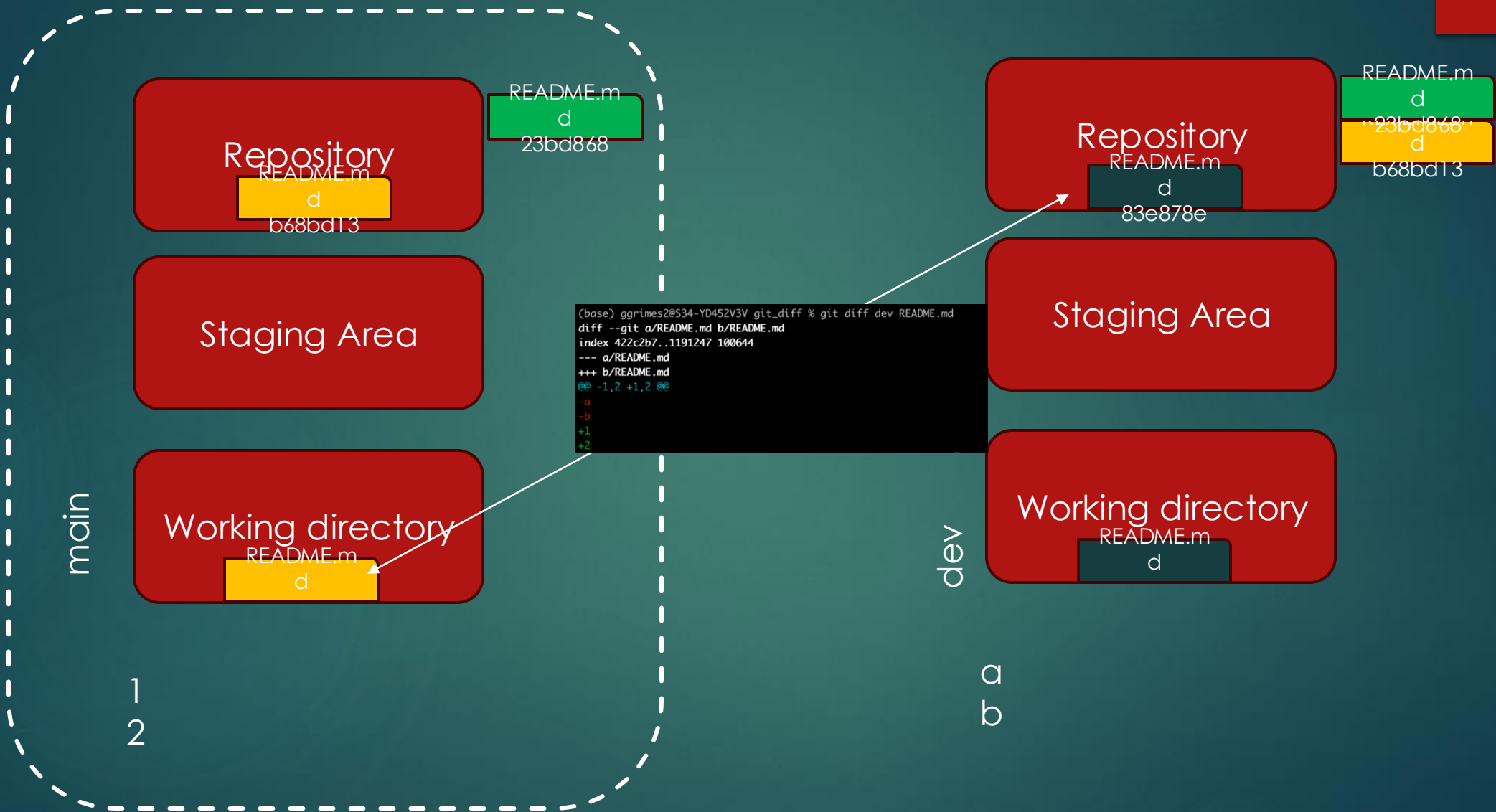
git commit -m "updated README.md with letters"



git checkout main



git diff dev README.md



git merge dev (fast-forward)

